UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI INGEGNERIA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

A web analytics framework based on the Lambda architecture

A.A 2014/2015

RELATORE

Cardellini Valeria

CANDIDATO

Luzzi Matteo Remo

CORRELATORE

Pedersen Glenn Skare
Aalén Thomas

Not all those who wander are lost - J.R.R. Tolkien

Contents

Acknowledgements

I would like to thank my supervisor Valeria Cardellini from the university of Rome Tor Vergata, for advising me in this thesis and for reviewing my work. I worked on this project as an intern at Vimond Media Solutions. A great thank goes to Glenn, Vegard and Kine for considering me a suitable candidate, to Helge and Andreas, and to anyone else who has helped me or has simply spent some time for a chat.

The Erasmus time had a relevant influence in my life. Of all the people that I've met, I would like to thank in particular Alessandra, Dario, Daniele, Elvira, Julia and Solvejg. Fantastic people, who so quickly became an important part of my life. I am not sure if without their presence I would have had that amazing time.

I want also to express my gratitude to my lifetime friends, Alessandro, Andrea, Giuseppe, Francesco, Lino, Manuel, Marta, Nicholas and Piergiorgio for all the crazy time spent together. Thanks for your patience and support, and for understanding my life choices even when they implied staying apart for long time.

Moreover, I would like to thank all of my university colleagues, for sharing with me the joyful and rewarding moments as well as delusions and hard times.

Finally, the greatest thank goes to my family and to my parents, Angelo and Gabriella. None of this could have ever been possible without their constant support. Thanks for always believing in me and for being an inspiration in every step I took in my life.

Abstract

From early 2000s, when the term Big Data was first coined, the evolution of technology has imposed new challenges on the design of data systems. The growing amount of data highlighted the limits of traditional architectures, therefore research moved its focus on new solutions able to handle it. Big data are also a remunerative market. It has been measured that, in 2013, the vendors revenue derived by related hardware, software and services was more than 18 million dollars, with a 58% boost respect to the previous year [29]. It is estimated that by 2017 the revenue will be more than 50 million dollars. Recently, we could notice an evolution for cloud-based big data services for large-scale analytics. More and more companies realized about the benefits of data analysis in order to carve out advantages in the market. A survey of 2013 [18] shows that more than 50% of the survey respondents declared that investing on analytics improved organization's competitive position, in terms of identifying new ways to perform sales and understanding users behavior for targeting them with ad-hoc campaigns.

Traditionally, for processing a high volume of data, a batch approach with tools like Hadoop is used, specially after the release of Yarn [44], which has broadened its applicability domains. However, some use cases, require to process data in nearly real-time, in order to have the possibility of taking business decisions quickly according to their results. What normally happens, is that these two ways of processing data are independent and are implemented in two different systems.

In response to a need of new approaches and patterns for big data processing, this thesis proposes a new architectural pattern: "VimondAnalytics", named after the company (Vimond Media Solutions [45]) where this master thesis project was carried out. It is based on the Lambda architecture, a pattern proposed by N. Marz [35], which provides a new way to combine the classic batch and real-time computations in one solution. Data are analyzed in two layers in parallel, batch and real-time respectively, and their results are stored on dedicated databases and finally merged in the serving layer. High latency of batch results is compensated by the real-time layer. This gives high availability to the system at the cost of eventual consistency according to the CAP theorem [14]. Even though Lambda architecture was first introduced in 2011, it has not been deeply investigated up to now, and we could find in literature only few examples of complete implementations [39], [9]. The purpose of this thesis is to provide a proof of concept implementation, and show its applicability in the fields of big data analytics. We will show how the initial pattern of Marz can be simplified, by eliminating intermediate databases, performing all the data aggregation and merging directly into the serving layer. The developed system has been deployed on AWS[1] (Amazon Web Service).
The results obtained are straightforward: having a safe data deletion mechanism leads to a decrease in the required time to query the system, compared to a classic solution where all data are ingested into the serving layer and never deleted. In case of throttled input frequency, we will show that VimondAnalytics query time is limited by an upper and a lower bound, depending on the chosen batch time window. We will also show how the system reacts to some failures of single components of the system. However, some of the results, are tightly coupled with the frameworks used for the implementation, and might not be valid outside this context.

[1] https://aws.amazon.com

The rest of this report is structured as follows:

- Chapter 1 gives an introduction of the context of this work, specifically about big data analytics and what is the state of the art in term of architectures and patterns for handling them.
- Chapter 2 briefly analyzes what are the tools and technologies used in this work, highlighting their main features and their architectures.
- Chapeter 3 describes VimondAnalytics architecture in detail. Great attention is given to each different layer and it is explained how they all cooperate in order to have a working solution.
- Chapter 4 discusses a possible implementation of VimondAnalytics using the tools introduced in chapter 2.
- Chapter 5 shows the results obtained with the proposed framework. General purpose tests were run in order to validate these results. For completion, the system was also tested under some common failover scenarios.
- Finally, chapter 6, summaries the work carried out in this thesis project. Furthermore, it outlines some future work scenarios in terms of enhancement of the current system features and applicability domains.

Keywords: Big data, analytics, Lambda architecture, batch processing, real-time processing, cloud computing, distributed systems

Chapter 1

Introduction

1.1 Background

In this section we will give an overview of the background this thesis project is based upon. We will briefly introduce the big data concept and its analytics derivation.Then we will focus then on the state of the art concerning software architectures for big data collection and analysis pointing out pros and cons of every analyzed solution.

1.1.1 Big Data

Big data is one of the trending terms in the IT field which increased of popularity from 2012 [24]. A lot of definitions has been given to it [46], focusing on different aspects regarding the data themselves and the infrastructures needed to process them.
Despite the recent dramatic increase of popularity, big data is a concept first coined in early 2000s. D. Laney introduced in his report a 3Vs concept concerning the new challenges that systems had to deal with, in terms of data features: Volume, Velocity, Variety [34]. The report focalized on the challenging issues to be addressed due to the increasing size of data, their frequency at which they are produced and a wider range of representation formats. Since 2001, when this report was published, the question about big data has not changed a lot. Now we have better approaches and technologies, but at

the same time the big data concept has evolved. We entered in the IoT era, where every device has internet access, from smartphones to electronic furnitures [7]. Hence, the massive amount of data needs new technologies for processing and analyzing it. A study from McKinsey Global Institute [37] stated that the amount of data will grow by 40% within 2020, and most likely this trend will not end soon. Big steps in technology has been made to handle this ever-growing data volume: for instance, traditional RDBMS have shown their limit in scalability, hence a NoSQL solution is generally preferred since the data load can be spread across several machines. Moreover, we switch from physical servers to cloud computing, where virtual resources can be requested on demand only when needed, avoiding for a company the burden of investing a lot of money in a static architecture solution [8]. According to what stated above, there is an endless need of new patterns and solutions for big data processing and the aim of this thesis is to investigate a new approach called **Lambda architecture** (Section 1.3.2) applied to a subgenre of it: big data web analytics.

1.2 Introduction to web analytics

The discipline of web analytics started in the early 1990s, as a consequence of the diffusion of public Internet [15]. Since then, monitoring users activity has become crucial, specially for business purposes, in order to improve efficiency and quality in organizations and services.

Almost every online business keeps track of their users, not only their actions (for instance visiting a certain webpage), but also information about devices used for accessing internet, connection speed, browser version and so on. Normally these information are then analyzed and used for user behaviour prediction and contents improving, like designing the user interface for a certain webpage [19]. Clickstream data is the primary

source of web analytics. It refers to the data collected from users activity, which are constituted by every kind of interactions with the website.
Nevertheless, web analytics scope is not enclosed into what stated above, typical use cases involve also social network analysis, security informatics or artificial intelligence [40], [23], [39]. Despite the use case, web analytics is just the first step in the overall process of data analysis: data need anyway to be understood by analysts for taking effective decision making strategies.

1.2.1 Web analytics metrics

To define appropriate indicators for web analytics is challenging. Big data became a powerful source of information only when a sense is given to them, hence standard terms and metrics are needed for analyzing them. Digital Analytics Association (DAA, formerly Web Analytics Association) was founded in 2004 with the mission of making analytics a professional discipline for the business, providing a set of common terms, definitions and best practices. A DAA document [47] from 2008 classifies web metrics in two categories:

- *count*: the most basic unit of measure. An example is the number of visitors for a web page;
- *ratio*: a derived metric, obtained dividing a metric by another one. An example is the number of page views per session.

A web metric can be *aggregate*, *segmented* or *individual*. An aggregate metric is applied to the totality of traffic for a website, a segmented one only to a subset of traffic, and an individual one refers to the one generated by a single user. Examples of them are: the average session time for the totality of the users, for the ones connected from a

particular nation and for a single user.

Even though DAA tried to standardize web analytics, there is still a certain degree of subjectivity when defining web metrics. A clear example is the concept of user session: for DAA a session ends after a logout event or a timeout of 30 minutes.On the other hand, Google Analytics [1] defines a session as a sequence of user actions valid till one of the following event occurs:

- the user stays idle for 30 minutes;
- day-change at midnight;
- the user changes the advertisement campaign for accessing the website.

1.2.2 A concrete example: Google Analytics

Google launched its analytics service in 2005 and released a free-to-use version one year later. It rapidly became one of the most used tools for website data analytics.

Google Analytics (GA) tracks the user behaviour through a combination of a Javascript library called *analytics.js* and cookies set on the user browser for its identification. What made this tool so famous is probably its usability, since no particular IT knowledge is required to set the tool up and running: a website owner can simply set via web console all the needed metrics. Figure 1.1 shows an example of report dashboard.

By default GA uses a javascript snippet called *google analytics tracking code (GATC)*, which is injected into web pages. As the browser loads a page as a consequence of an HTTP request, GATC is activated and requests the *analytics.js* library from Google servers, needed for performing the actual data collection via tracker objects. HTTP

[1] https://support.google.com/analytics/answer/2731565?hl=it

[2] http://ppchero.wpengine.netdna-cdn.com/wp-content/uploads/2012/05/dashboard-.png

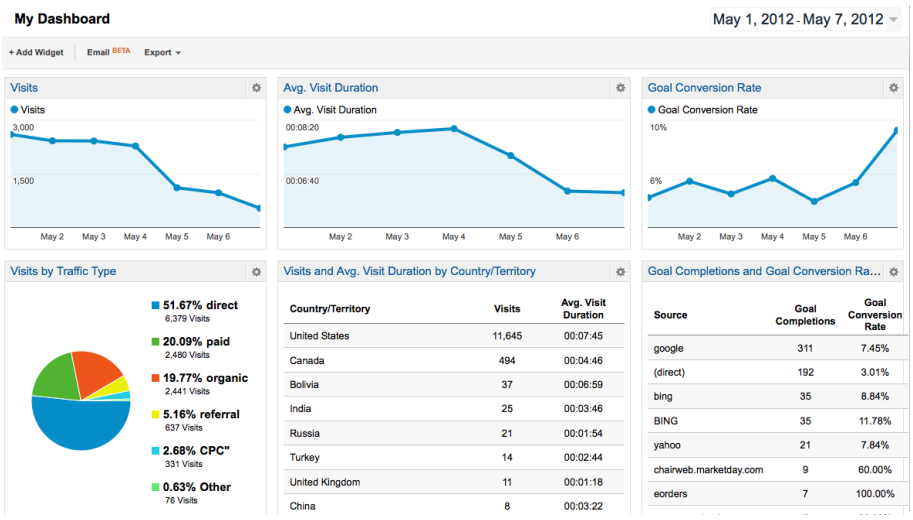


Figure 1.1: Google Analytics dashboard.[2]

requests contain valuable information about the clients executing the requests themselves, such as user agent, hostname, origin and language. In addition, *analytics.js* sets and reads *first-party* cookies to keep track of the current user. After the tracking has been completed, data are sent to the GA servers as a list of attached parameters to a single-pixel GIF image request for being processed and analyzed.

There are several issues related to GA which can mine its accuracy. The first is about the cookie usage: setting a cookie identifies a particular user when data are sent to the server for being analyzed. If the user deletes the cookies set by the analytics library, it will be considered as a new user by the server, so altering the analysis (for instance when tracking the number of unique visitors on a web page). Secondly, GA uses sampling, which means that not all the clickstream is gathered and analyzed, but only a sample of it. This can clearly lead to a drop in final results accuracy.

1.3 State of the art for data system

As stated above, dealing with analytics data could be misleading, and sometimes *key performance indicators* could be interpreted in different ways. Therefore, many companies prefer to build their own in-house analytics system rather than using a commercial one (like GA). Analytics systems are a particular case of Big Data systems: hence, when designing and developing them, we can apply the same theoretical background [28]. Distributed systems are normally preferred in place of standalone architectures as they offer better performances due to scalability, but they also have some drawbacks: they are complex and less consistent due to the distributed nature. Moreover, when design a platform for Big Data analysis, one has to bear in mind the following factors: data size, speed or throughput optimization and model developement [41]. For instance, if data size fits in the system memory, a single machine is often enough to process them, otherwise a cluster solution should be used.

Currently, Big Data systems fall into these two categories:

- *batch processing* of big volume of data with high latency;
- *real-time processing* with short latency (milliseconds) of a stream of data.

Batch solutions have been very popular in the last years, specially after Google's publication about *Map-Reduce* computation paradigm [17]. The general approach consists into a set intermediate of key-value pairs generated after a *map* function, and a final set of key-value pairs as a result of a *reduce* function applied to intermediate sets, which are aggregated by the same keys. The aim is to merge input values in order to generate a possibly smaller set of values. Map-Reduce provides a high level of abstraction over a parallel and distributed computation: a programmer only needs to define a map and a reduce function and as a result the computation is spread across several nodes in a

cluster. A classic example of this is counting the occurrences of each word in a document. A map function for this problem takes in input a line of the document and emits a tuple for each word in the line. The corresponding reduce function only counts the number of tuple received for a key (tuples are distributed across the reducer by key). There exist several opensource solutions inspired to Map-Reduce, being Hadoop[3] the most famous one.

Real-time computation has been introduced to overcome the high latency which characterizes the execution of a batch process. Data are considered as a stream and the goal is to process them as fast as possible. This approach is often coupled with business intelligence and data mining processes. Stream processing systems can process data independently or look for complex patterns into the data stream itself. While the first approach is stateless, the latter implies looking at the past, hence previous states must be persisted either in memory or in a external storage solution [28]. An interesting overview [16] defines real-time processing system as a natural evolution of traditional DBMSs. Even if there are substantial differences, as for instance that data need to be stored before being processed, both systems process incoming data with a sequence of transformation based on common SQL operations defined by relational algebra like *joins*, *filters* or *selections*.

A third minor class of data processing handles big data *in-memory*. A study from 2012 [21] predicts the in-memory processing will soon overcome the classical batch approach. The report highlights the benefits of having a fast data processing system, especially when it comes to react fast to new data with decision making processes. Of course one of the drawbacks of this approach is the cost: memory is 10 times more expensive that disks, hence companies should go for such a system only when really

[3]http://hadoop.apache.org

needed. Another possible issue is related to the dimension of the data to be analyzed since a dataset could be to big to be kept in memory. In these situations, a batch approach is generally preferred.
However, modern applications often require multiple processing models in their workflows, specially in the analytics domain. Real-time analysis of the click stream of a web page followed by batch processing to compute clicks correlation could be such an example [28].

1.3.1 CAP theorem limitation

CAP (Consistency-Availability-Partition) theorem states that, in every system, only two of the three previous features can be achieved. In presence of distributed systems we are forced to have network partitions, then the CAP theorem says that we could only tradeoff consistency over availability or vice versa. This means that we can either have a system capable of always serving user requests sacrificing consistency, or a system that sometime is not available, but gives out consistent data (with all the partitions). Keeping this in mind, we will explore the state of the art for the class of system architectures which aims to mitigate the effects of this theorem.

1.3.2 Lambda Architecture

Since data system are particularly influenced by CAP theorem, a different approach is needed in order to find a tradeoff between availability and consistency. As stated before, a system can perform batch or (nearly) real-time operations on its data. We will explain in the next sections how the Lambda architecture aims to find a balance between them.

Motivations and architecture

Lambda architecture is an architectural pattern for big data processing proposed by Nathan Marz [35]. It aims at addressing to the problem of querying a data system, giving result *as much as possible synchronized* with the input data. This implies a user to be able to query the system in real-time in order to obtain information on the current state but also on historical data.
According to its manifesto [35], this type of architecture has to fulfil the following requirements:

- robustness and fault tolerance against hardware failures and human mistakes;
- low latency of read and update operations;
- scalability;
- generalization in terms of different use cases covered by the system;
- extensibility, i.e. the ability of adding new features easily;
- ad-hoc queries support.

The purpose of each data system can be summarized into the simple equation $query = function(alldata)$ [35] with the complication that, dealing with big data (gigabyte/petabytes), implies a considerable amount of time for executing a query. In this case, its result could not be consistent with the entire dataset, since while computing it, new data might have been injected into the system. Unfortunately, there is no such a single system capable to answer to every kind of query *in real-time*.
The first concept introduced by Lambda architecture is to pre-compute query functions [36] in order to process them quicky. If a user queried a particular timeframe

of data, the system would immediately answer by showing the precomputed result. Computing aggregated results imposes high latency as well: a typical instance of aggregation deals with data collected in the last hours or even days and it may take hours to complete. Then, the last step then, is to fill the gap between aggregated results and new data not covered by aggregations yet.

In order to solve this problem, Lambda architecture proposes a three layers pattern composed by:

- Batch layer;
- Real-time layer;
- Serving layer.

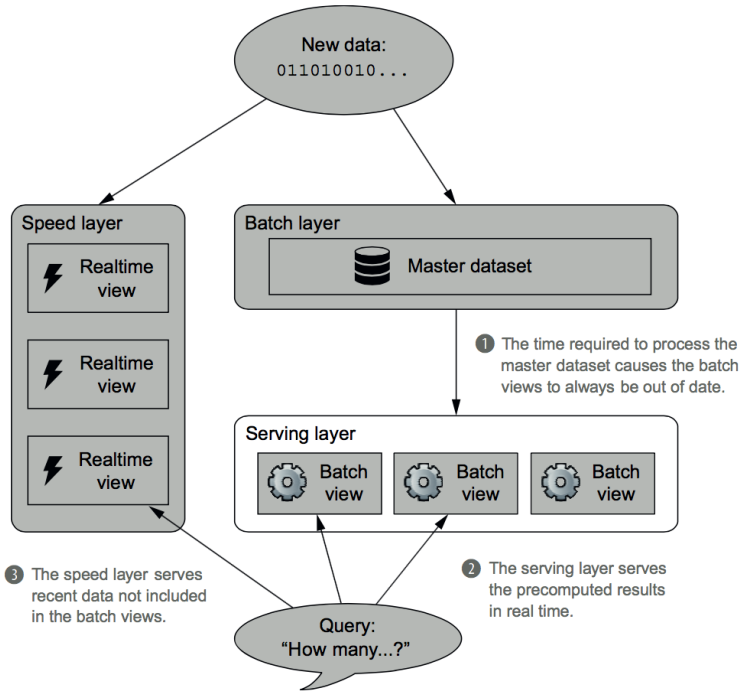


Figure 1.2: Lambda architecture as union of three layers [36]

Every single datum ingested into the system is replicated into and elaborated by both the batch and real-time layers in order to compute intermediate results (we will refer to them as *batch* and *real-time views*. The outcomes are then merged into the serving layer and ready to be queried by end-users. Each layer has different characteristics according to the role it fulfills in the overall architecture.

The batch layer is composed by two parts: a master dataset and a batch computation part. The master dataset holds the data coming into the system. Data are considered immutable, no updates or deletion are in theory allowed: data can be appended into or read from the master dataset. We will show later how this is a great advantage in terms of consistency. The batch computation part reads from the master dataset and computes batch views. According to Marz [36], the bath layer would solve the equation $batch\,view = function(all\,data)$. For the latter operation two approaches are possible: incremental and recomputation. The first one implies an update of previous views in order to compute the current one, while the second involves a total recomputation based only on the raw data stored in the master dataset. Recomputation algorithms are easier as they result in a simpler logic, while sometimes using incremental algorithms may speed up the whole process as it exploits the existence of already computed results. The choice of the best approach really depends on the use cases the system has to address to. For instance, let us assume we want to calculate the total number of visitors for a webpage over the time. It is easy to understand that, for this use case, an incremental approach is preferred over a total recomputation on the entire dataset which is executed everytime new data come in.

The real-time layer aims at filling the gap introduced by the high latency of the batch layer and it is responsible to provide results on the most recent data. It takes the same input of the batch layer, but rather than of storing it in a dataset, elaborates it

on-the-fly. It behaves as the batch layer, in terms of pre-computing the same type of views, the difference is that it works only with a small portion of data, exactly the one which it is not covered by batch views. An incremental approach is generally preferred and real-time views are updated as new data come into the system. Formally, it solves the equation $realtime\,view = function(realtime\,view, new\,data)$ [36]. Overall, the design of the real-time layer is more complex of the batch layer, both for its incremental approach and for the requirements it has to respect such as short latency. Fortunately, it is only responsible for a small dataset and its views are eventually corrected by the batch ones, so normally a certain degree of approximation is tolerated. We will go more in detail into this in the next section.

The serving layer is responsible for providing results to ad-hoc queries with low latency as a combination of batch and real-time views. It indexes results from the batch layer in order to serve them with low latency and it compensates the lack of batch views for the new data by querying the real-time ones. Hence, it gives a final answer to the problem of querying a data system by solving the equation $query = function(batch\,view, real\text{-}time\,view)$ [36]. It is the layer with the most complex logic as it has to merge different views according to type of queries submitted to the system. A simple case is when the data are time ordered and a particular query refers to a fixed timeframe: the serving layer has just to aggregate batch views with real-time ones (if needed). However, this is just a possible use case and a general purpose implementation of the serving layer might be not trivial.

CAP theorem and Lambda architecture

Notwithstanding the audacious title of Marz article about lambda architecture (*i.e. "How to beat CAP theorem"* [35]), CAP theorem does still hold [25]. What Lambda architecture aims to achieve is the so-called eventual consistency. This means, that, after a certain timeframe, data are eventually consistent across all the partitions of a distributed system.

We stated before that every single layer of the Lamdba architecture has to respect different constraints due to their different role in the overall system. The master dataset has only to support create and read operations, making then its data an immutable source. This is a great advantage, in the sense that the system does not have to deal with updates, which are the cause of consistency lack in distributed systems. The overall batch layer is eventually consistent in the sense new data are consistent with the query results for a time interval only after the corresponding batch views have been calculated. Moreover, a database supporting random reads and batch writes is used for storing such views.

Instead, the real-time layer, has to be higly available in order to compensate the latency introduced by the batch layer. In order to compute real-time views quickly, approximation algorithms are often used, rather than exact ones (for instance, the evalutation of distinct records of a dataset is generally estimated using *hyperloglog* algorithm [27]) and the database used to store the real-time views has to support both random writes and reads due to the incremental approach used by this computation.

The overall Lambda architecture based system, as we said, is eventually consistent. Approximations and mistakes made by the real-time layer are eventually corrected by the batch views, once they are computed.

Pros and Cons

One of the main problems the Lambda architecture wanted to solve is the ability of querying data in real-time. In a certain way, this goal has been achieved: the high latency introduced by batch computation is compensated by real-time views and the system is always able to respond to a query, with a certain degree of accuracy as previously stated. This is indeed a great result, especially compared to what we can achieve with regular data systems (real-time or batch computation).

Having an immutable source of data gives to the system robustness against failover: if an error happens on the batch layer (either an error in the processing logic or a hardware failure), the entire computation can be re-started without any data loss. Calculation of the final result as a series of transformations of an immutable dataset is a concept also used in batch/micro batch computation frameworks like Spark [48].

The main weakness of such a system is without any doubts its complexity. The serving layer has the most complex logic as we explained before, especially in case of non trivial queries. Also, under a development point of view, implementing and maintaining two different systems is hard. Unfortunately, at the time it does not yet exist a unified framework for sharing the same logic between the real-time and batch layers. An attempt of unification has been made with Twitter summingbird project [13]. It provides a high level abstraction for defining the logic of a job and then sharing it among different frameworks for batch and real-time computation. Nevertheless, this is still a raw project rather than a production-ready solution. Another option would be using a module of the Spark ecosystem called Spark Streaming[4], even though it just simulates a real-time approach by actually performing *micro-batching*. Another drawback is the lack of flexibility regarding the queries the system can serve. One of

[4]http://spark.apache.org/streaming

the requirements of the Lambda architecture is the ability to handle *ad-hoc* queries, in the sense they depend on parameters only known at query-time. Instead, it lacks is the ability to serve arbitrary queries since it deals only with precomputed views.

1.3.3 Alternatives to Lambda architecture

The Lambda architecture is not the perfect approach, other solutions and proof concept ideas found in literature try to exploit its weaknesses.

Kappa Architecture

The main competitor of Lambda architecture is an architectural pattern called *Kappa architecture* [30], first mentioned by Jay Kreps (project leader of Kafka at Linkedin) in 2014. This approach claims to beat Lambda architecture in its main weakness: the complexity of the overall system. Kreps says that maintaining such a complex system likely leads to lack of synchronization between the real-time and the batch layer as well as a high effort for running and debugging it.

The solution proposed by Kappa architecture is to exploit just the real-time computation layer, working with a system which is able to persist messages over the time (Apache Kafka, for instance). In the Lambda architecture, the real-time and batch layers share the same logic even though they work on replicated dataset. In the Kappa architecture, approach data are stored and reprocessed several times by the same stream processing logic, according to the different time views required by the system. Each processed result is then stored in a serving layer in charge to switch between several views depending of the queries executed by end-users. In this scenario, multiple processing processes can work in parallel on the same dataset by reading it from different points. For instance, one coud be processing data from one month ago, one from one week ago and another one could just follow the real-time flow of new data.

Real-time Streaming architecture

Stream processing is a popular solution when systems have to address use cases of low latency computation. Many companies are moving their processing architecture from a batch approach to a real-time one, due to the maturity of recent stream processors, which are able to handle more complex workloads [38]. Moreover, several datasets are event-based, for instance generated by system logs, and hence more suitable for a real-time processing paradigm rather than a batch one. Real-time systems can also be

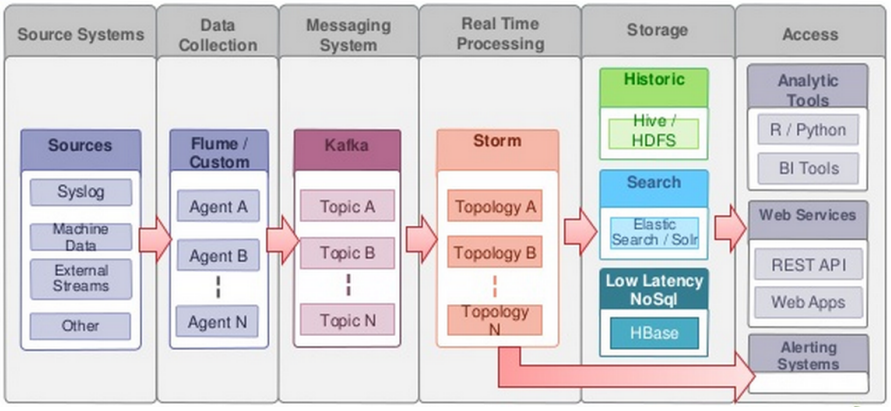


Figure 1.3: Real-time architecture.[5]

used as ETL (extracts, transforms, loads) engines for a batch processing architecture. As shown in Figure 1.3, real-time processing could be just the first step in the overall data analysis process. Its output can be either used as it is, giving real-time insights, or can feed searching or historical data processing modules, like data warehouse. The advantage is that data have already been cleaned, transformed and augmented before starting the real analysis process. Indeed, one of the features a real-time processing system should have is the capability of integrating stored and streaming data [42].

The literature offers several real-time streaming systems implementations, both open-

[5]http://www.slideshare.net/Hadoop_Summit/design-patterns-for-real-time-streaming-data-analytics

source and proprietary. Although they have differences, they are all based on the same processing model, which considers data as an unbounded sequence of discrete elements to be processed on a DAG node structure. The most famous platforms are IBM InfoSphere Streams [33] and Apache Storm [6]. Streaming processing frameworks are continuously evolving in order to address new challenges in Big Data computation. Twitter has just replaced its production solution Storm with Heron [42], with improvements in terms of debuggability, cluster resources allocation and multi-applications execution. A new interesting platform, Apache Flink [1], has been designed to provide a unique solution both for batch and real-time processing, sharing the same core concept of the Kappa architecture: a powerful stream processor that is able to process both real-time and historical data.

IoT architecture

When talking about IoT architecture, we refer to a new way of handling billions of different data sources. Normally, behind every source (for instance, smartphones and laptop) there is a physical person. Instead, in the IoT paradigm, small systems like sensors generate data autonomously [7]. Cloud computing based architectures are too centralized for handling such a geographically distributed source of data. Fog computing [12] aims at extending the classic cloud approach with a new framework for IoT computing, having data analytics as one of its key use cases. Recalling the 3 Vs paradigm [34], Fog computing deals with a fourth dimension of Big Data, which is their geo-distribution at the edges of the system.

Fog computing can be seen more as an extension of the Lambda architecture applied the case of IoT, rather than an alternative to it. Fog computing has a hierarchical architecture having at its core the classic cloud paradigm. The rest of the architecture

is composed by a set of end-points located at the edegs of the system, which are connected with the core through distributed network resources. Moreover, Fog computing enables distributed deployment of applications on nodes located at different points of the system, hiding the heterogeneity of the infrastructure by exposing a common interface for resources allocation and management.

Data coming from sensors are actionable in real-time, and in addition, they can be used to run analytics processes over longer periods, which take place in a normal cloud computing environment.

Chapter 2

Big Data tools

In this chapter we will describe the tools that can be used for implementing a working prototype based on the Lambda architecture. The scope of this chapter is not to make a complete survey of modern technologies for big data but rather to provide only an overview of them, highlighting them purpose and main features of each used tool. For a better understanding of these tools, the interested reader can consult additional resources about them, as listed in the bibliography.

2.1 Kafka

Apache Kafka [31] is a general purpose, distributed, publish-subscribe message system originally developed at Linkedin, open-sourced in 2011 and inserted in the Apache ecosystem. The architecture is composed by one or more servers, also known as *brokers*, which rely on Apache Zookeeper for coordination.

The key abstraction in Kafka is a topic: *producers* publish messages to a topic and *consumers* subscribe to one or more topics. A single topic is divided into a set of partitions, distributed and replicated among all the brokers in the cluster. For each partition, there is a master replica and a set of slaves. Since there are usually more partitions than brokers, a broker can play the role of master for some partitions and

the role of slave for others. Moreover, metadata about consumers positions (also called offsets) are saved for each partition. An offset represents a checkpoint for that consumer: messages before it are assumed to be correctly processed and the next time a consumer makes a fetch request, the broker will forward messages starting from the last offset. Anyway, the consumer, is still able to reprocess already committed messages, for instance in response to a failover event. Figure 2.1 shows a producer appending messages to a 2-partition topic. At the same time a consumer is reading from the same topic according to its offsets.

An interesting feature of Kafka is how messages are forwarded from producers to consumers through the brokers. Unlike other messaging systems which process messages in-memory, Kafka does a strong usage of the filesystem: messages are immediately stored on the filesystem and not deleted when read by a consumer, but retained according to a configurable period. This avoid any kind of data loss for every committed message, even in case of broker failures. The only information retained by brokers is the consumers positions across the topics. These information are generally stored on Zookeeper.

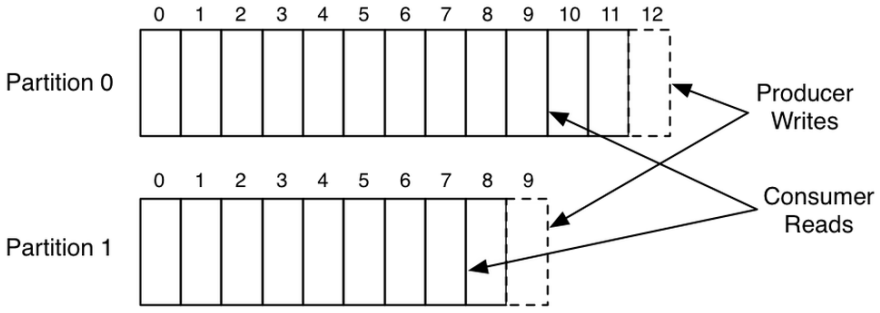


Figure 2.1: A 2-partitions topic with one producer and one consumer.[1]

[1]https://engineering.linkedin.com/kafka/benchmarking-apache-kafka-2-million-writes-second-three-cheap-machines

Kafka allows to implement either a *queuing* or a *publish-subscribe* logic introducing the abstraction of a *consumer group*. Each consumer labels itself with a group name and is it responsible for reading from one or more partitions of a topic. Kafka ensures than no more than one consumer per group reads from a partition: if in the group there are more consumers than partitions, some of them will stay idle. In the opposite scenario, a consumer will read from more than one partition. If all the consumers belong to the same group, then the system works as a queue, balancing the load across the consumer group. If all the consumers have different groups, then the system works as a standard publish-subscribe. Figure 2.2 shows the consumer group concept. Two consumer threads belonging to the same group equally share a 4-partition topic, reading from 2 partitions each. Instead, in case of a group with 4 consumer threads, each of them reads from only one partition.

Having the concept of the consumer group implies a total order over the messages

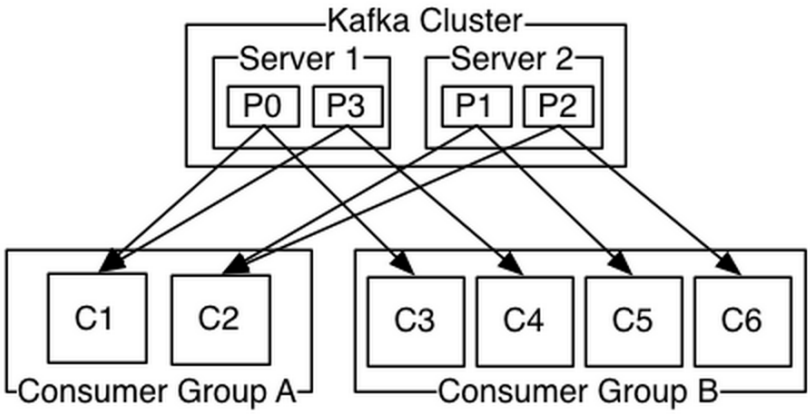


Figure 2.2: Two consumer groups read from a 4-partition topic.[2]

within each partition: the order of how messages are read inside a partition is the same for every consumer who subscribed that partition.

[2] http://kafka.apache.org/documentation.html

As a conclusion, it is worth mentioning that Kafka implements by default an *at-least-once* message semantic in the following way:

- Producer: when publishing a message and having a network error, the producer cannot be sure if the message has been correctly committed to the broker.
- Consumer: a message is said to be fully processed (from a broker perspective) only when its offset has been committed. The best the consumer can do to read the message, process it and then commit the offset. If the consumer crashes before committing the offset but after processing the message, the one taking over will process the same message.

2.2 Hadoop

Hadoop [2] is an open-source suite for scalable fault-tolerant distributed computing written in Java. Based on Google's *Map-Reduce* [17] approach, it was first developed in order to run MapReduce jobs for processing a web crawl. The rapid increase of popularity led to an abuse of the original computing paradigm exposed by Hadoop, such as run heterogeneous tasks in a multi-tenant environment. In 2013, a second version of Hadoop, YARN [44], was released with the purpose of compensating the limitations of Hadoop first version, in primis by decoupling the programming model from the resource management layer and supporting a multi-tenancy job scheduler.
The architecture of YARN is composed by a global per-cluster *Resource Manager (RM)* with duties of scheduling the jobs submitted by clients and allocating resources when needed by the applications running on the cluster (Figure 2.3). Having a centralized coordinator enables scheduling properties of fairness, capacity and locality. The archi-

[3]http://hadoop.apache.org/docs/r2.7.0/hadoop-yarn/hadoop-yarn-site/YARN.html

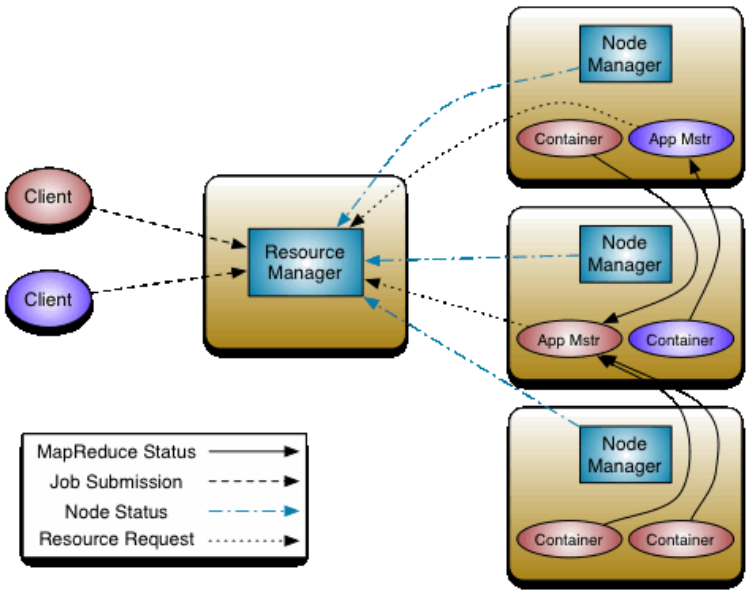


Figure 2.3: YARN architecture.[3]

tecture is completed by one or more *Node Managers (NM)* running on each slave node in the cluster. NMs are the workers daemons, in charge of monitoring the execution of the applications running on their nodes. Resources are spawned across running applications in form of *containers*. A per-application daemon called *Application Master (AM)* dynamically requests the RM bundles of resources (containers) for executing the application it is responsible of. In this scenario, the RM is agnostic of the applications logic as it only satisfies to resources requests. MapReduce is now only a class of applications Hadoop YARN supports and several computation frameworks implementing different programming models have been adapted for being executed on top of YARN (Spark, Storm, Tez, Giraph and others).

2.2.1 HDFS

Hadoop distributed storage implementation is called HDFS (Hadoop Distributed File System). It is a highly efficient storage solution, perfectly integrated with applica-

tions running on Hadoop, supporting the main operations of a normal filesystem and structured in a folder hierarchy. Its architecture resembles that of the Google File System [22] and it is composed by a master/slave architecture with by one *Namenode* and at least one *Datanode*, as shown in Figure 2.4. A Namenode holds the namespace

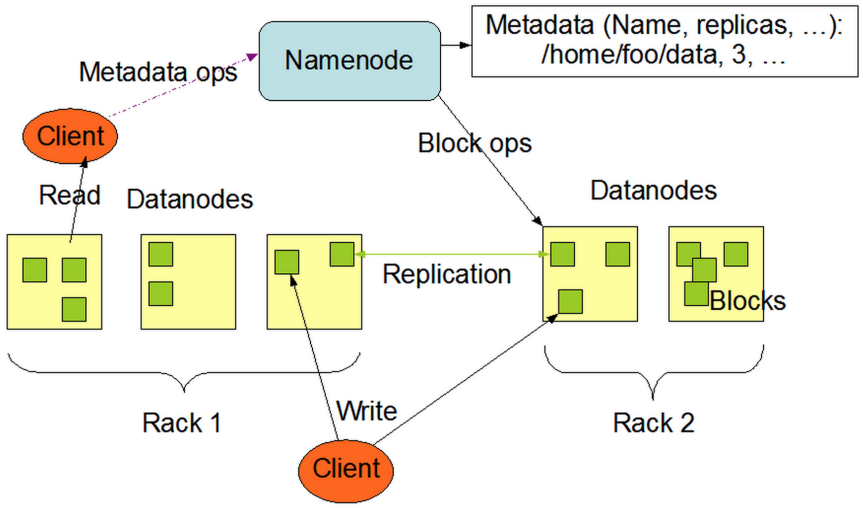


Figure 2.4: HDFS architecture.[4]

of the filesystem, i.e., information about the structure of the filesystem and metadata about the distribution of the files over the datanodes, and listens for client requests such as writes or reads. Datanodes are the worker nodes: they store data and serve the actual client requests, providing data in case of read requests or allocating space in case of write requests.

A file in HDFS is split into several blocks according to its size (HDFS handles block of 128Mb by default), distributed and replicated across the Datanodes according to a per-file policy. The position of each blocks and their replicas are kept in Namenode's namespace. Having a block abstraction gives robustness and fail-tolerance to the storage system in case of Datanodes failure: client requests are redirected to the nearest

[4]http://hadoop.apache.org/docs/r2.7.0/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

healthy Datanode containing the required information.

2.2.2 Oozie

Apache Oozie [4] is a workflow scheduler for Hadoop. Each workflow is composed by a sequence of action executed in sequential mode, which means that an action cannot be executed until the previous one ended. If configured with YARN resource negotiator, Oozie runs each job in a container context (Section 2.2). Oozie workflow builds a DAG (direct acyclic graph) structure and is composed by a sequence of two possible types of *nodes*:

- control flow: nodes that control the start, the end and the execution of the workflow without executing any concrete action. Example of these nodes are: start control node, end control node, kill control node, fork control node.
- action: nodes that execute a task. The actions supported by the latest version (4.2.0 July 2015) are Map-Reduce, Pig, Java, Fs (filesystem), ssh and Spark.

Jobs are scheduled by the server with a minimal granularity of minutes. It is possible to specify an input and an output directory where to read data needed by the job and where to store a possible result. Moreover, the input and the output directory paths can be expressed in a dynamic way using time expressions, which are evaluated differently according to when the job is executed. All the configurations are submitted to the server using XML files containing information about the location of the application executables, directory paths, application arguments and jobs frequency. These configuration files are uploaded on HDFS in order to be accessible from any node in the cluster.

2.3 Spark

Apache Spark [5] is a distributed framework for batch data processing. It is based on the same paradigm of Hadoop/Map-Reduce, where the computation is split across several nodes in the cluster (*map*) and the final result is given by aggregating the intermediate ones (*reduce*). Spark has been designed for that class of applications which need to iterate several times on the same dataset, such as machine learning and interactive analytics, allowing them to store intermediate data in memory rather than recomputing them at every step. For these types of applications, Spark outperforms a classic Map-Reduce paradigm by 10 times [49].

In order to be executed in a distributed mode, Spark needs a cluster manager for resources allocation across all the nodes. Spark provides its own standalone cluster manager, even though YARN or Mesos [3] can be used. The architecture is composed by a primary node called *driver program* and a set of worker nodes running *executor* threads (Figure 2.5). When an application is submitted to the cluster, the driver program spreads it across the worker nodes and schedules task to be run by each executor in an independent context (each executor runs its own JVM). In this way, there is a complete isolation between several applications running on the same cluster. The drawback is that, executors belonging to different applications cannot share data between themselves and also within the same application. They need the support of the driver program for sharing data by using special objects such as *broadcast* and *accumulator* variables.

The main concept around Spark is an abstraction over a physical dataset called *RDD* (resilient distributed dataset) [48]. RDDs are read-only, fault-tolerant parallel data structures divided into one or more partitions across a set of machines. They can be

[5] https://spark.apache.org/docs/latest/cluster-overview.html

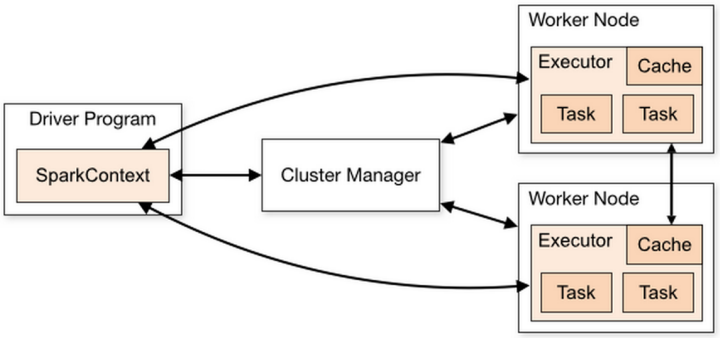


Figure 2.5: Spark cluster architecture.[5]

created from physical files, parallelizing a collection of elements or modifying an existing RDD by applying a transformation (map, filter...) or an action (reduce, count...). RDD are lazy by nature, in the sense that the driver program keeps in memory the list of transformations needed to compute it, rather than executing them at each step. Only when an action requires the RDD to be returned to the driver program, the list of transformations is applied. By default, each transformed RDD is re-calculated each time an action runs on it. However, the transformed RDD can be cached in memory for faster successive accesses. This abstraction increases the robustness of the data model: in case of a partition failure, it can be reconstructed by applying the list of transformations and actions from the last RDD available in memory.

2.4 Storm

Apache Storm [6] [43] is a distributed real-time data processing framework created by Nathan Marz (the proposer of Lambda architecture). Storm considers the data as a stream of *Tuple*, generated in special component called *spouts* and processed by one or more *bolts*.

Spouts and bolts are combined in a DAG structure called *topology*. Each component

(spout or bolt) can be replicated for load distribution.

A Storm cluster is composed by a master node and one or more worker nodes. The master node runs a process, called *Nimbus*, which handles applications submissions and it is responsible for distributing the code across the nodes, assigning tasks and checking for failures. The worker nodes run one or more processes called *supervisors* in charge of communicating with the master and executing the worker processes. The communication between *Nimbus* and each *supervisor* relies on *Zookeeper* (Figure 2.6).

Within a worker process it is possible to run one or more *executor* threads. Finally,

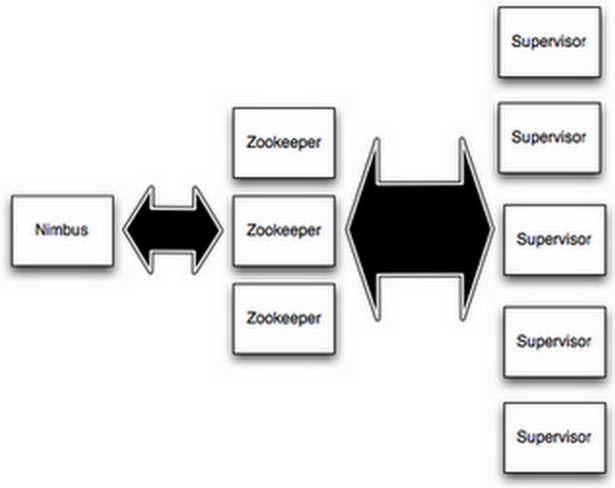


Figure 2.6: Storm cluster architecture.[6]

each *executor* runs one or more tasks, with are just parts of the topology belonging to the same component. As stated before, the main abstraction in Storm is the stream of Tuples. Data flows from a component to the next one according to a partition policy: data can be distributed in shuffle mode, by a key, towards a unique component or totally replicated. Each mode fulfils a different use case: for instance a key-partitioning (*fields grouping*) is useful when we want to count the occurrences of a particular el-

[6]https://storm.apache.org/documentation/Tutorial.html

ement in the data stream, while a shuffle partitioning (*shuffle grouping*) is a better approach when the same operation can be performed on each tuple, independently of the actual content of the tuple itself.

Storm guarantees by default the *at-least-once* message processing semantic. A topology can be launched with acker components, ensuring that each tuple is successfully processed within a timeout. Storm keeps in memory a DAG for each tuple generated by a spout. Each node in the DAG represents a component which is supposed to process the tuple and sends an ack on success. If the acker receives all the acks for a tuple, then that tuple is said to be fully processed, otherwise the tuple is considered failed and the spout responsible for it should be able to regenerate it. In this way, no data loss is guaranteed, but a tuple might be processed more than once.

2.5 Elasticsearch

Elasticsearch [20] is a distributed full-text search engine built on top of Apache Lucene. It provides a set of REST API for documents indexing, querying and real-time analytics. In order to give a better understanding of how documents are organized inside a cluster, it is possible to compare the Elasticsearch structure to a relational database where:

- a database is called *index*;
- a table is called *type*;
- a table row is a *document* in JSON format;
- a table column is a *field*.

Elasticsearch uses Lucene's inverted index data structure for fast full-text searches. Basically, an inverted index stores, for every unique word inside each document, a list

of records containing that particular word. Furthermore, each word can be normalized according to a text analyzer: for instance, it can be converted to a lower case, or in a singular form, if it is a verb it can be converted to the infinitive tense, or words can be aggregated if they are synonyms. This step is crucial for having a fast search engine and the user can tune it according to his use cases.

Elasticsearch provides a simple master/slaves cluster architecture: either a master or a slave node is capable of indexing or querying documents, the only difference is that a master node is responsible for coordinating operations such a creating an index or adding/deleting a node in the cluster.

It is worth mentioning that an index is just an abstraction for one or more physical data space called *shard*. At index creation time, the user can specify in how many shards to divide it, and how many replicas for a single shards. This is an important setting since the number of shards cannot be dynamically changed as long as the index is active, only the number of replicas is dynamic. Every time a node leaves or joins the cluster, the master node balances the shardes and the respective replicas over the actual cluster composition.

Chapter 3

Architectural design of VimondAnalytics

The aim of ths chapter is to describe the architectural design of the proposed framework for Big Data analytics. First we will briefly introduce Vimond Media Solutions the company indicating how this thesis adress its need in terms of lack of an analytics module. Then we will describe each layer of the proposed solution.

3.1 Case: Vimond Media Solution

Vimond Media Solution [45], the company where this thesis project was carried out, is an OTT (Over-the-top) services provider company headquartered in Bergen, Norway. With customers spread around the world, Vimond is one of the leader in the streaming media market. It provides for a suite of software application called Vimond Media Platform which helps content owners, broadcasters and distributors to manage their online video services. It provides top features such as REST API, multi-tenancy support, multi-language metadata and much more.

3.1.1 Vimond current architecture

The Vimond architecture (shown in Figure 3.1) is based of a set of microservices communicating with each other via a publish-subscribe module implemented with Apache Kafka. Using a microservices-based architecture improves modularity and scalability of the whole platform. Content providers interact through the system with a component called Vimond Control Center, which allows them to ingest, manage and distribute their assets. End users access those assets via an API abstraction over the microservices, after being authenticated with O-Auth protocol. At the time of writing, relevant actions such as subscription purchase or asset playing performed by end-users are just stored into Cassandra and Oracle databases and used only when needed, for instance when customers request usage reports. What is missing is an independent module for gathering and analyzing these data, possibly in real-time with support for historical data. The Lambda architecture seems perfect for defining such a module. In the rest of this chapter we will show and describe the proposed framework we have realized.

3.2 Proposed solution

The proposed solution is a proof of concept of the Lambda architecture, rather then an effective production-ready system. It aims at improving the high-level idea of the Lambda architecture by simplifying it where there is the majority of problems (i.e., the complexity), still resulting in a three layer architectural pattern. In the rest of this thesis we will refer to the proposed solution as VimondAnalytics.

The main simplification comes when dealing with intermediate views. According to the original Lambda architecture pattern, each processing layer produces intermediate views to be merged into global ones in the serving layer. The model we are going to

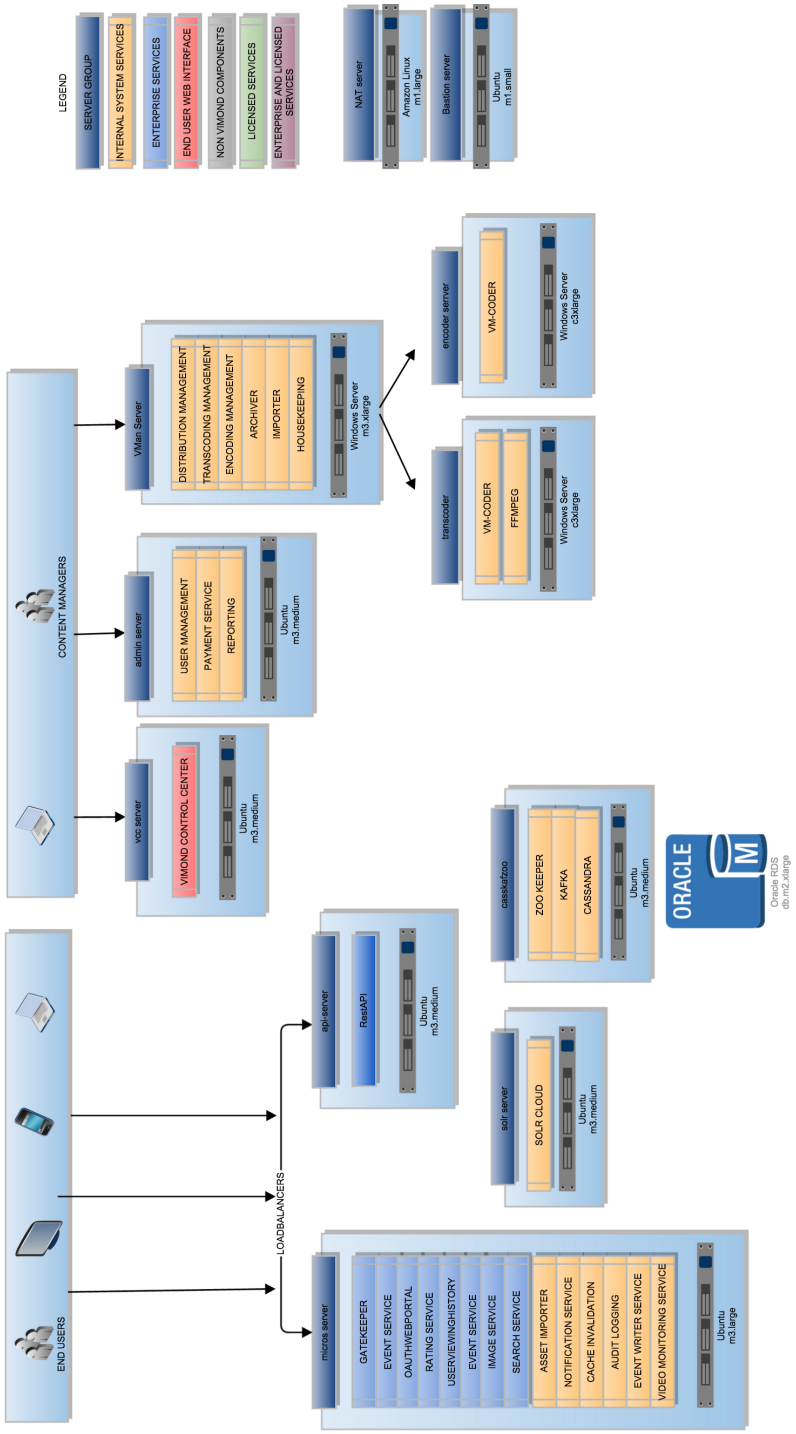


Figure 3.1: Example of Vimond production environment.

propose avoid storing intermediate views in different databases: data are ingested in the serving layer. The purpose of the latter is similar to the original one as it has to merge data coming from the two different processing layers. However, instead of views, it now merges raw data coming from the real-layer and aggregated ones from the batch layer. The simplification is hence obvious: there is no need anymore for different storage solutions which have different requirements and therefore different problems. All the complexity is moved to the serving layer, while the real-time and the batch layer fulfill only to processing requirements. Figure 3.2 summaries the high level architecture of VimondAnalytics

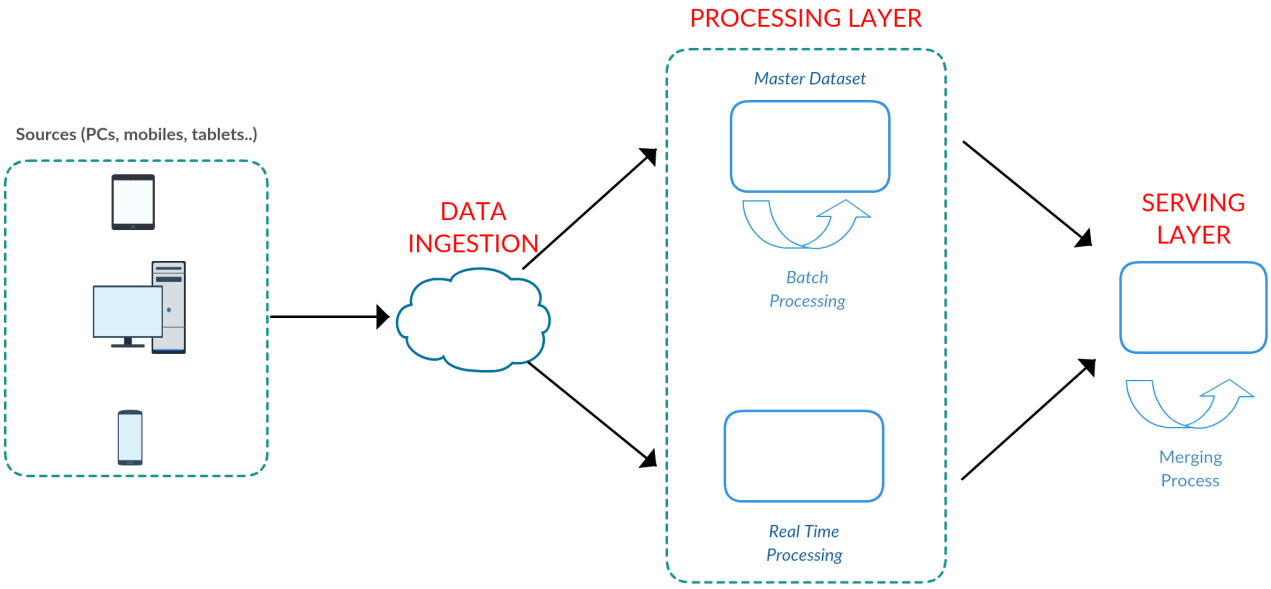


Figure 3.2: VimondAnalytics architecture

The final architecture is again a composition of several layers. Data come into the system through a *data ingestion* layer, implemented with a publish/subscribe architecture. The *real-time layer* gets data from the data ingestion and, after few processing steps, stores them in the serving layer. The *batch layer* stores data in a data lake structure

and runs aggregation processes for obtaining compact (and shorter) views out of them, which will be eventually stored as well in the serving layer. The latter aggregates raw data and merges them with the batch ones in order to answer to end-users queries.
The rest of the chapter describes in detail each separate layer of the model we have so far introduced.

3.2.1 Data ingestion

The data ingestion layer represents the entry point of the system, where input data are managed and replicated into the batch and the real-time layers. Apache Kafka fits perfectly into this role, as it handles different consumers and has them fetching messages from different points on the same dataset, according to their capability.
In order to make the whole system working, we need a data source. Artifical datasets are commonly used for a proof of concept validation but we rather preferred to use a real one. Vimond provides TV2 Norway with services for its online streaming platform called *TV2 Sumo*. Data used in this thesis come from that platform and refers to a player playback functionality which allows a user to resume the stream of an asset when he closes the browser window and wants to re-play from the last point.

3.2.2 Real-time layer

The real-time layer of VimondAnalytics is very simple, being its purpose to forward data from the data ingestion part to the serving layer after having processed them. Differently from the original approach, it does not have to perform any data aggregation or data storing on an intermediate database. A possible action executed in this layer could be data augmentation, like IP translation into geographic location and co-

ordinates or user agent broken into browser and operative system version. Neither an incremental nor a recomputation algoritm is used: augmented data are just written on the serving layer. A typical example of data forwarded by the real-time layer is:

```
{
    "eventType": "click"
    "userId" : "12345",
    "targetURL" : "http://www.mywebsite.com/mypage.html"
    "timestamp": "2015-09-06-19:54:00"
}
```

Listing 1: RealTime data example

To implement this layer we used Apache Storm as it provides the features we need. No message guarantee policy is required, we are not interested in having a strong consistency level, data need to be moved as fast as possible and a certain degree of inaccuracy is tolerated. Anyway, real-time processing frameworks such as Storm allow to implement an *at-least-once* message processing rather than *at-most-once* where messages loss is possible. According to the use cases of the current scenario, it is possible to switch to the needed semantic.

3.2.3 Batch layer

The batch layer is organized in two parts. First, data are fetched from the data ingestion layer and saved into a data lake, then they are analyzed and aggregated according to the output views.

In order to improve the performances of the fetching process, data are stored in the data lake in batches and we designed two ways to accomplish this task:

- **unreliable**: the offset for a message is committed after it has been read from the data ingestion layer. This implies that, if the process crashes or fails to write

the message on the data lake, the message is lost. In addition, if messages are processed in batch, in case of crash of the process, all the batch is lost. This approach implements a *best-effort* message delivery semantic, being unreliable.

- **reliable**: the offset is committed after a batch of messages is written successfully on the data lake. In case process crash, the one who takes over is going to read from the last committed offset which represents the latest checkpoint. In case of process crash after having written the messages but before committing the offset, data will be written twice. This approach implements an *at-least-once* message delivery semantic.

The data lake is implemented through Hadoop HDFS. As explained in Chapter 2, HDFS implements an abstraction over a distributed data storage such that classic filesystem operations are possible. Data written into the data lake are organized in a time-folders structure: the dataset is a hierarchy of folders where at the top there are the ones representing days, the second level is for the hours of a particular day and the third is for the division of the hour in intervals. It is possible to divide an hour or a day into several intervals but partitions bigger than a day were not implemented. When a new datum needs to be written in the dataset, it goes to the corresponding folder according to its timestamp. For instance, let us assume a partition criterion of 15 minutes: every folder representing an hour contains 4 sub-folders representing the minutes intervals (00-14, 15-29, 30-44, 45-59). Figure 3.3 explains where data are actually written in this scenario, according to their timestamp.

While new data are ingested into the data lake, the batch layer is responsible for executing the aggregation processes on the historical ones. Each of these processes takes as input the data contained inside a specific folder, which changes over the different

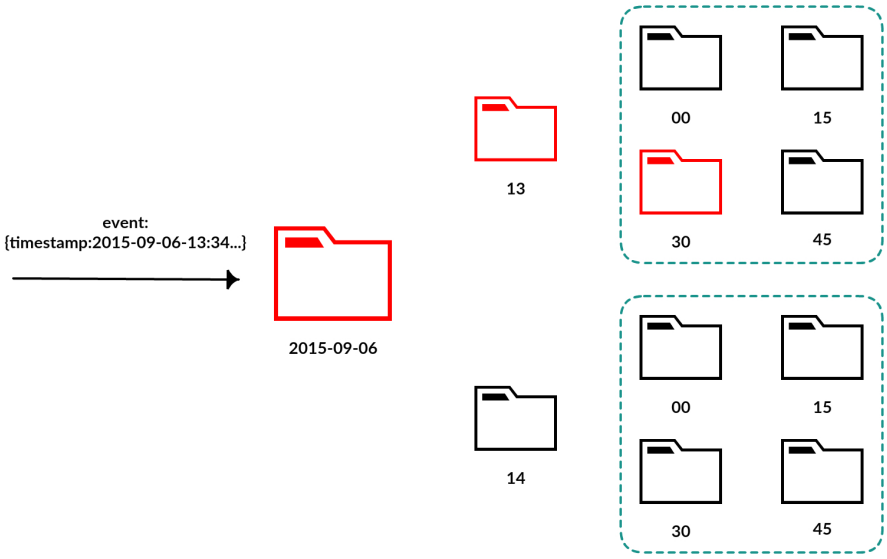


Figure 3.3: Time folder structure: an event is saved in the corresponding folder, according to its timestamp.

executions, and produces as output a set of events representing aggregated data, which are then directly into the serving layer, without the use of an intermediate database. A typical example of a datum generated by this layer is:

```
{
    "eventType": "click"
    "targetURL" : "http://www.mywebsite.com/mypage.html"
    "counter": 100
    "timestamp": "2015-09-06-19:54:00"
}
```

Listing 2: Batch layer data example

An incremental approach can be used, but this model has been thought for a recomputational approach over the different data subsets. The different batch views produced at each execution can be aggregated with each others in the serving layer. While this approach simplifies a lot the whole processes, it has limitations in the type of results

it can generate as explained in Section 3.2.5.

Finally, after a batch window of data has been processed, according to the principles of the Lambda architecture, real-time data included in that time window are no longer needed as we have an aggregated version, much smaller and faster to query. It is a duty of this layer ensuring that, after the succesful completion of a batch process, the corresponding real-time data are deleted. In this way, all the approximations made by the real-time layer are eventually corrected by the batch views.

3.2.4 Serving layer

The serving layer of VimondAnalytics presents the biggest differences with the original idea proposed by the Lambda architecture. As observed in Chapter 2, the serving layer should index the batch views and merge them with the real-time ones. On the contrary, in VimondAnalytics, there is not anymore the concept of intermediate views, but only final ones are presented to end-users. To this purpose, we need a solution which provides storage and aggregation functionalities.

Two kinds of data are written into the serving layer:

- data coming from the real-time layer;
- aggregated data coming from the batch layer.

According to the data examples provided in Figure 1 and Figure 2 a serving view regarding the count of clicks for a particular web page merges all the real-time data for that webpage with the ones generate from the batch layer, according to the timeframe requested by the query. If the serving layer uses JSON syntax as well, then an output result could be the one presented in Listing 3. Elasticsearch was used for implementing this layer. We are not interested in the full-text search engine but we exploited mostly its aggregation functionalities on the documents stored into it.

```
{
    "eventType": "click"
    "targetURL" : "http://www.mywebsite.com/mypage.html"
    "counter": 200
    "from": "2015-09-06-19:00:00"
    "to": "2015-09-06-20:00:00"
}
```

Listing 3: Serving Layer data example

3.2.5 Considerations

In this chapter we presented VimondAnalytics, a Lambda architecture based model for big data analytics. The idea behind this model is the simplification of the general pattern, removing intermediate views and then complexity from the whole system and computing final views in the serving layer, as a combination of real-time raw data and batch results.

Concerning the batch layer, looking at the data in time windows causes limitations in the type of results that can be overall computed. Let us assume to have two consecutive batch windows of data, created by a time partitioning criterion as explained in Section 3.2.3. The different results calculated on each window are then stored in the serving layer and aggregated with each other if a query wants to extract data from a time interval containing both of them. Hence, we need to ensure that the aggregated results are consistent among all the different batches, which is, given two partitions A and B and an aggregation function f, the following equation $f(A) + f(B) = f(A + B)$ must hold. For all the other cases where it does not hold, an incremental approach is needed. Examples of consistent views are generated by unique events over the time, like a purchase or a content rating.

Other limitations of this approach are inherited from the Lambda architecture: it is not possible to query the system for a time interval shorter than the batch time window. The smaller the granularity, the greater the accuracy of the query, but computing batch jobs more frequently might not be the best option in terms of efficiency, even if applied to a smaller subset of data.

The applicability of this framework lies on the use cases where a normal query would investigate what happened in a fixed time span in the past, corresponding to one or more batch windows (for instance, recalling the previous example, the number of clicks for a webpage from 10 am to 12 am, assuming a 30 minutes batch interval), or a real-time overview over all the historical data saved in the serving layer.

Chapter 4

Implementation of VimondAnalytics

In this chapter we will present a concrete implementation of the VimondAnalytics architecture described in Chapter 3. For each layer we will provide explanation of the most important features using diagrams and code snippets.

The whole system has been implemented in Java. Our choice was driven by the frameworks used for the implementation: Storm is implemented in *Clojure* but it provides Java API, Spark and Kafka are implemented in *Scala* but they do offer as well Java API as Elasticsearch does. Finally, in some parts, past projects from Vimond have been used or modified, and since Vimond Core API are written in Java, no other language choice was possible.

4.1 Data ingestion

TV2 runs Sumo production environment into a virtual private cloud on Amazon Web Service (AWS), isolated from the outside network, hence it is not possible to directly fetch its data. A bastion server is usually placed as protection against undesired accesses, in such a way that only trusted users can access the private cloud and network traffic is routed through NAT instances.

Since Sumo platform uses Apache Kafka as a message sharing system between several services, it has been convenient and easy to create a "data bridge" (Figure 4.1) inside that environment for pushing data to VimondAnalytics. For this purpose we decided to use an already existing Vimond project called "Firehose", which has been modified for this use case. Firehose simply implements a data connection between two endpoints. Data are read from a Kafka topic, validated and forwarded to another Kafka cluster. For security reason, Firehose is installed on a machine inside TV2 VPC: in this way the source Kafka cluster is directly accessible through private IPs. A public IP (AWS elastic IP) is assigned to the VimondAnalytics Kafka cluster, in order to be reachable from outside the private VPC.

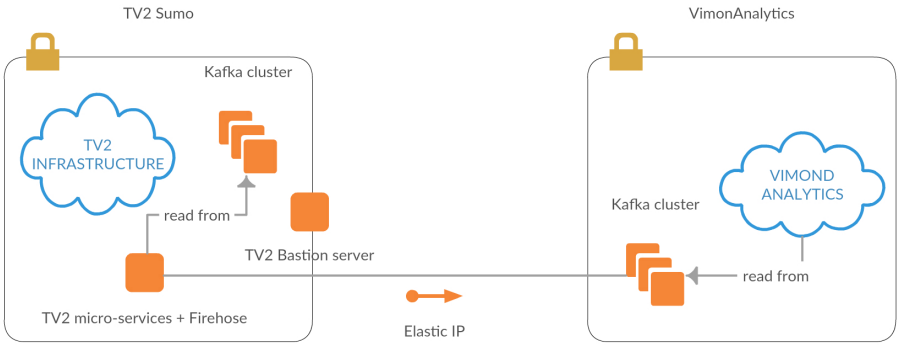


Figure 4.1: Firehose data bridge

Data model

As said in the previous chapter, the data used for this project come from the resume player functionality provided by TV2 pay-per-view platform *Sumo*. Data are generated by players running on different devices: this results in two different type of data events according to whether the player uses Silverlight[1] plugin or not. The data events generated by other plugins contain only a subset of information of those generated by

[1]http://www.microsoft.com/silverlight

Silverlight plugin. Specifically, an example of non-Silverlight event is:

```
{
    "eventName":"player_log_event"
    "originator":"player"
    "timestamp":"2015-09-13T10:31:39.879Z"
    "guid":"5d33aa50-8232-447e-8b5a-14b4317f1265"
    "progress":
    {
        "assetId":958291
        "title":"God morgen Norge"
        "offset":
        {
            "duration":7133249
            "pos":7101143
        }
        "category":
        {
            "id":919990
            "name":"Innev?rende sesong"
        }
    }
}
```

Listing 4: Non-Silverlight event example

In addition to this information, an event generated by Silverlight plugin contains another field with details about the client used for playing a particular asset. In Listing 5 we reported the most important fields used in our analysis.

```
{
    "client":
    {
        "playerEvent":"period"
        "userAgent":"Mozilla/5.0 (Windows NT 6.1; WOW64; rv:40.0)
        Gecko/20100101 Firefox/40.0"
        "env.platform":"silverlight"
        "buildName":"Vimond.Player.TV2Sumo"
        "video.format":"smil"
    }
}
```

Listing 5: Silverlight event example

Each event is handled by the different layers either as a simple Java String object or serialized into a custom *StormEvent* or *SparkEvent* object when its internal state has to be accessed or modified.

4.2 Output results

One of the limits of the Lambda architecture is that the output needs to be defined a priori. According to the considerations we made in Section 3.2.5, the following output views have been defined:

1. Number of times of starting or ending asset.
2. Most started assets.
3. Most ended assets.

4. Most browsers used when starting an asset.

5. Most OSes used when starting an asset.

6. Most video formats used when starting an asset.

In order to have consistent outputs across the different batches, we exploit the field *"playerEvents"* which gives information of the event type generated by the player reproducing an asset. The type can assume *start*, *period* and *end* values. We could not use the event marked with *period* as the different batch windows do not share information and then every result produced out of them would have not been consistent among different time windows. A *playerEvent=start* or *playerEvent=end* represents instead a unique event. Unfortunately, only a subset of the input data contains information about the player event (i.e., the one generated by the Silverlight plugin). A request has been forwarded to TV2 for having this information also in the events generated by other devices. Moreover, it has been requested to change the mode of generating *end* events: a user likely stops the streaming of an asset before it actually ends, for instance when skipping the ending credits, and this results in a smaller number of this kind of events. An alternative solution would be to consider an asset to be ended when the streaming reached a certain percentage over its total length, for instance 95%. These features are going to be implemented within the next TV2 *Sumo* platform release.

4.3 Batch layer

4.3.1 EventFetcher

EventFetcher is the process in charge of reading events from Kafka and storing them in the HDFS according to their timestamp. There are two different execution modes:

unreliable and reliable. Figure 4.2 shows the class diagram for the main classes involved, reporting the most important fields and methods.

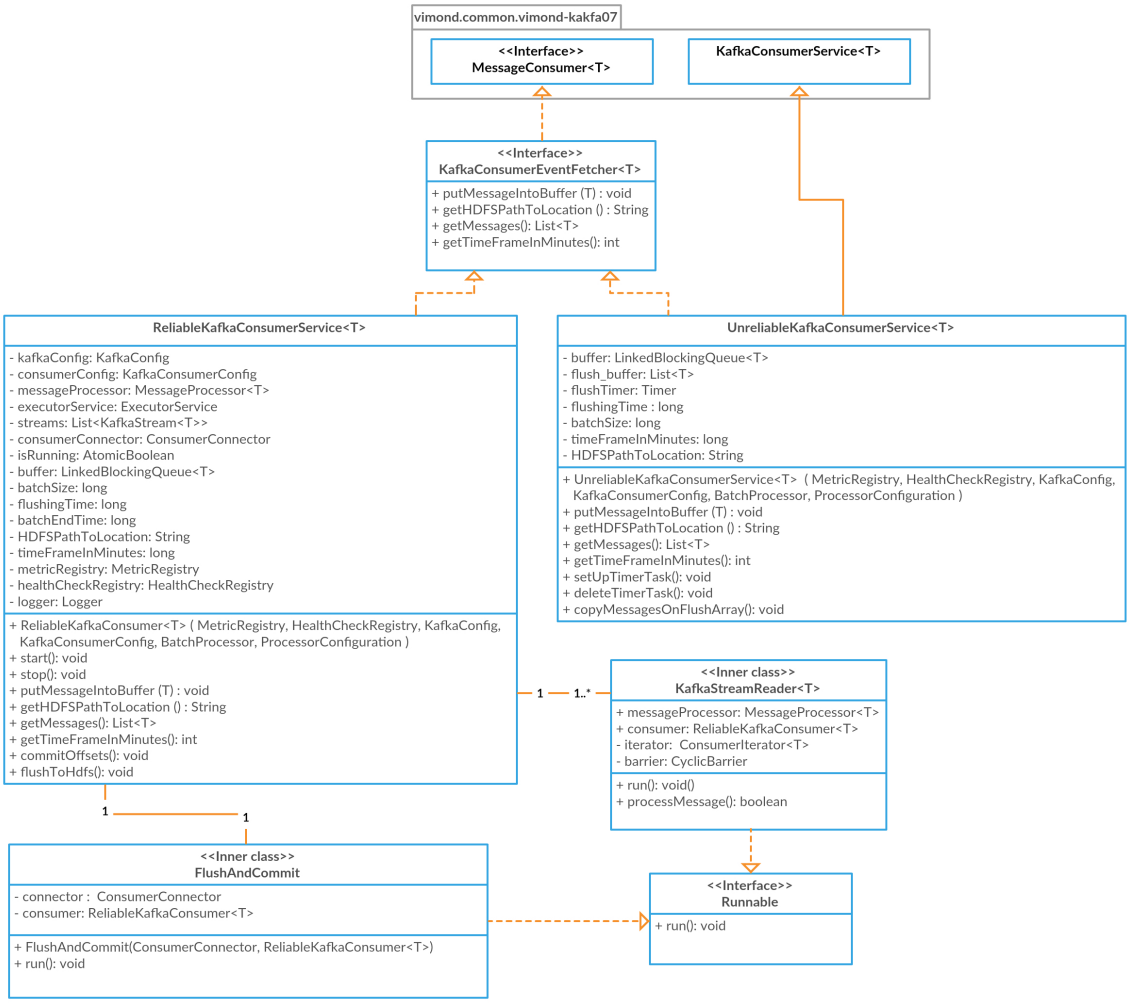


Figure 4.2: EventFetcher class diagram

The *UnreliableKafkaConsumerService* extends a Kafka consumer class defined in the Vimond repository since the latter offers methods for reading messages from a brokers and committing the offset immediately after. What was missing was the ability to process the messages in batches. Each consumer threads reads from Kafka and puts the message in a *LinkedBlockingQueue*. The usage of this object is due to its thread-safeness. Each batch operation is triggered either by time or by a threshold

on the number of messages stored in memory. Whenever one of these events occurs, the messages are copied into a separate buffer and written in batch into the HDFS from a separate task. that message writing is not a bottleneck in the overall process. However, it does not give any guarantees about the correct execution of the operation, because the consumer threads are not aware of the writing one, and a failure of the latter would not be detected by the others.

The *ReliableKafkaConsumerService* behaves similarly to the unreliable counterpart, however it does not commit a message offset after reading it but only when a batch is successfully written on HDFS. All the consumer threads are synchronized by a *CyclicBarrier* object. Every time a message is received, it is stored in a common buffer. Then, each consumer thread checks if the message threshold or a batch interval has been reached and, in case, blocks itself on the barrier. The last thread to complete these operations is that delegated to write the messages into the HDFS: it first tries to contact the HDFS Namenode and, if the latter is available, it proceeds with the writing part. In case the Namenode is not available, for instance due to a network error, it keeps retrying to contact it using an exponential backoff policy. When the writing process is completed, the offset is finally committed since that batch of messages is considered processed. In order to avoid deadlock, the other threads wait on the barrier for no more than a configurable time; if the timeout expires, the overall programs raise an exception and exits. This is not the best approach, even though it guarantees that the next execution of the program reads the same messages that otherwise could have been lost.

In both the modes of executions, data are written as binary files according to the *SequenceFile* format defined by Hadoop.

4.3.2 Batch data pipeline

To implement the batch data pipeline, we decided to use Apache Spark. Spark itself does not have an internal jobs scheduler, hence an extenal one is needed to re-execute the same batch job, each time parameterized in a different way. Considering the usage of Hadoop HDFS as a solution for the data storage, using Apache Oozie, the official job scheduler over Hadoop, resulted a convenient solution. As we explained in Chapter 2, Oozie allows to schedule a DAG of actions, where an action is executed only when the previous one ended.

Figure 4.3 shows the sequence of actions performed at every execution:

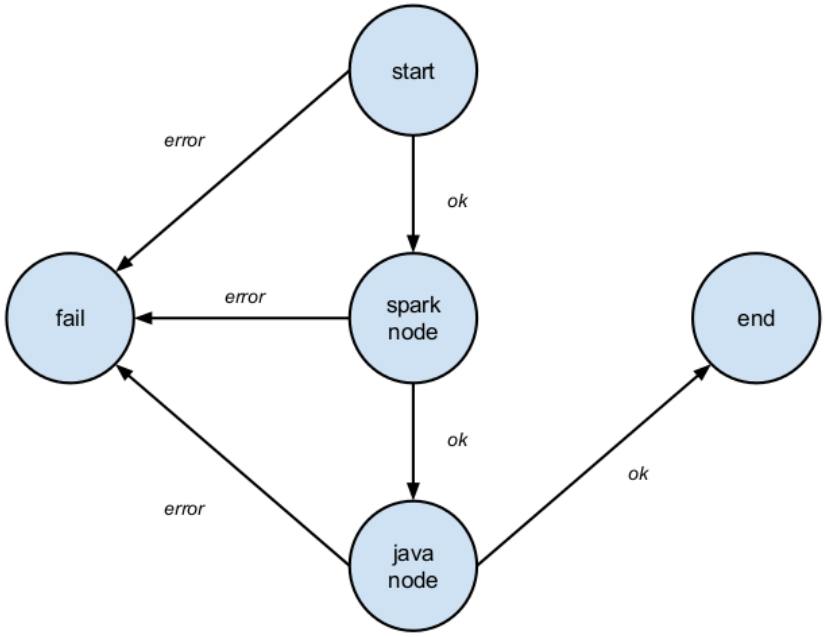


Figure 4.3: Oozie data pipeline

- start, fail and end are control nodes;
- spark-node and java-node are action nodes.

The user specifies the input parameters such as the beginning time or the frequency of the execution through a property file containing a list of *key=value* tuple. A single execution of the DAG is defined through an XML file called *workflow.xml*, while the jobs scheduling is defined with a *coordinator.xml* file. Oozie allows to define parameterized variables which are resolved with a different value according to the current execution. In our implementation, a job takes a different input folder according to the time of the execution, as showed in Listing 6.

```
<datasets>
        <dataset name="vimondEvents" frequency="${coord:minutes(30)}"
        initial-instance="${datasetStartTime}" timezone="GMT+02:00">
        <uri-template>${nameNode}/user/${coord:user()}/${datasetFolder}
        /master/${YEAR}-${MONTH}-${DAY}/${HOUR}/${MINUTE}</uri-template>
        </dataset>
</datasets>
<input-events>
         <data-in name="input" dataset="vimondEvents">
                  <instance>${coord:current(-2)} </instance>
        </data-in>
</input-events>
```

Listing 6: RealTime data example

With the *dataset* tag it is possible to define a parametric path where to read the data according to the *YEAR-MONTH-DAY/HOUR/MINUTE* pattern. A dataset can be an input or output one. In this case, with the tag *data-in* we consider an input dataset resolved into the folder filled with two batch windows before the current execution. For instance, let assume to have a 30 minutes batch window and let the next be executed at 17 pm of a certain day: that job will work on the dataset containing the data from 16:00 to 16:30. This is a complication derived from Kafka: since it does not ensure messages ordering, we cannot be sure that at the time of the execution of a job, all the events of the previous timeframe have been written into the correct folder. We

have checked empirically that, delaying the execution of a job of two time windows, guarantees that a batch executions reads all the data is supposed to work with.
After the validation of the input parameters, the execution continues through the spark node. This is the step where raw data are actually analyzed and aggregated in order to have a compact view over them. Since we already use Hadoop, in order to avoid the deployment of another processing platform, we decided to run Spark jobs on Yarn. As stated in Chapter 2, Yarn is independent from the computing paradigm as it only allocates resources to containers which make requests for them. There are two modes of running Spark on Yarn: *client-mode* and *cluster-mode*. In the first one, Spark driver program is run on the client, while in the latter the driver runs within the same container of the Application master. We used the *cluster-mode* and considering the case of having two executors, this results in three containers allocated by Yarn, two for the executors when the actual processing happens and one for the driver which just coordinates the process. The following picture represents the steps executed within the program.

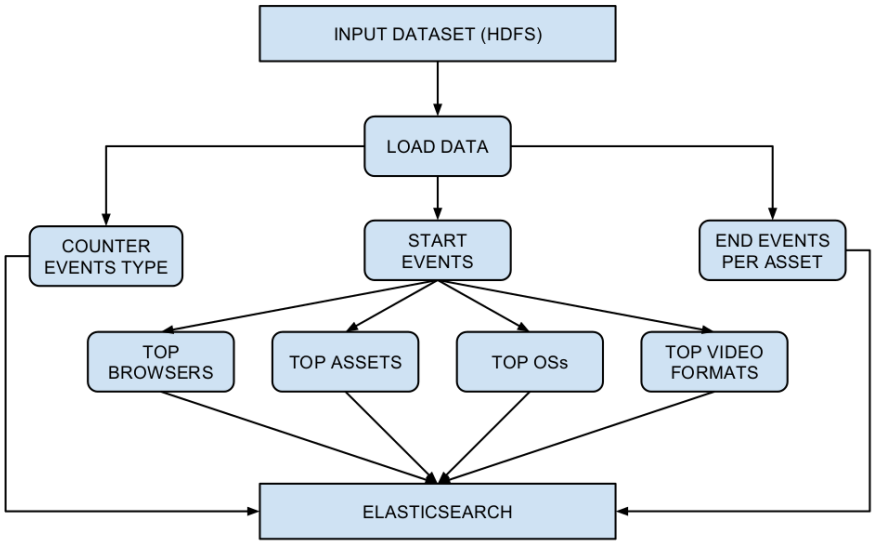


Figure 4.4: Spark actions

Spark provides methods for loading binary data directly from HDFS, so no effort is required for converting byte arrays in Strings. Every loaded event is then converted into a *SparkEvent* object for being analyzed. Since Kafka offers an *at-least-one* message semantic, there might be duplicates, hence an additional filter process is required in order to remove them. Unfortunately, this is the most expensive step as it has to analyze the whole dataset, and usually shuffle operations between the executors are needed. The dataset obtained in this step (LOAD DATA block) is then cached in memory since it is needed by every other process yet to be run in the current execution. In case of a dataset bigger than the available memory, it is spilled to the disk and deleted when it is no longer needed. Anyway, this leads to a drastic performance drop.
From this dataset, three branches are executed:

- Counter events type: this step calculates the number of start and end events;
- End events per asset: this step calculates how the end events are distributed among the assets, i.e., what are the assets that users watch till the end;
- Start events: this step is just an additional filter over the loaded dataset. Only player start events are considered. From the output of this process four additional steps are executed to calculate what are the most used browsers, operative systems, video formats and assets.

The aggregation processes have a rather simple logic as they are composed only of map and reduce operations over the dataset. For completion, we report a pseudocode of the process that calculates the number of player-end events per asset.

Algorithm 1: End events per asset aggregation

```
Input: AllEvents input
Result: Aggregated data written on the serving layer
begin
    endEvents ← emptyRDD()
    foreach event e in input do
        if e.playerEventType == end then
            endEvents.add(e)
    pairEvents ← emptyRDD()
    foreach event e in endEvents do
        tuple ← (event.assetName, 1)
        pairEvents.add(tuple)
    reducedEvents ← emptyRDD()
    foreach tuple x, y in pairEvents do
        if x.assetName == y.assetName then
            reducedTuple ← (x.assetName, x.counter + y.counter)
            reducedEvents.add(reducedTuple)
    outputData ← emptyRDD()
    foreach tuple x in reducedEvents do
        output ← createBatchData(x)
    writeDataOnServingLayer(outputData)
```

This is just an example of how output data are generated, the process itself does not iterate several times on all the datasets, but leverages the distributed power provided by Spark. A dataset is divided into several partitions and distributed on all the working processes that execute the pseudocode above illustrated on a smaller sub dataset, fetching data from other partitions only when needed. By default, the number of partitions of an input file is equal to the number of blocks used by HDFS to store it. Then, an optimization is to ensure that each file on HDFS is slightly smaller than an HDFS block size in order to avoid creating partitions with just few data.

All the branches eventually write data on the serving layer in form of JSON documents. The batch output data model is composed by a field indicating the event type, one for the subject of the aggregation and another one for the aggregated value. For instance

an event representing the most used browser is:

```
{
    "eventName":"TopBrowser"
    "data.browser":"Firefox 40"
    "counter":"1202"
    "originator":"VimondAnalytics"
    "tenant":"tv2"
    "timestamp":"2015-09-20T10:20:23"
    "timewindow":"10"
}
```

Listing 7: Top browsers output event

Figure 4.5 shows the class diagram for the most important classes involved in the aggregation process. An aggregation action has been mapped into a Java object implementing the *Job* interface. In addition, for the jobs using an input dataset to work with, an abstract class named *WorkingJob* has been defined. Finally, jobs are started through a *JobStarter* object which uses a factory pattern for jobs creation.

The last step of the batch pipeline is the deletion of the real-time data after the completion of a batch aggregation process over a timewindow of data. This is simply obtained by querying the serving layer for the data contained in the current time interval and then deleting the corresponding results. The serving layer implemented with Elasticsearch does not support standard SQL-like delete operations as *DELETE from table WHERE condition*, but it requires the *ids* of the documents to be deleted. This process is done in batches in order to improve the throughput of the operations as it would be network expensive sending a delete request for each document. The overall process goes under the name of *delete-by-query*.

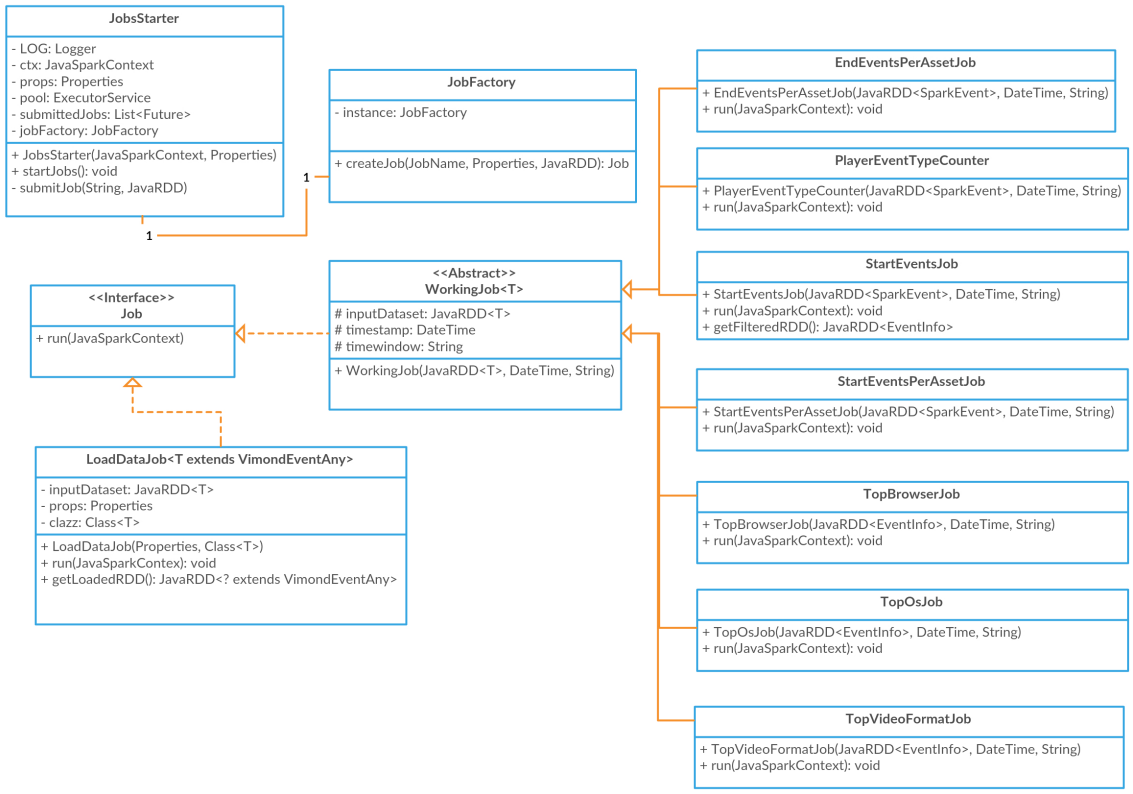


Figure 4.5: Batch class diagram

4.4 Real time layer

Storm topology runs in parallel with the batch layer and propagates data from Kafka into Elasticsearch. It is a rather simple topology as it computes just a data augmentation step and no aggregation at all.

Data are fetched by Kafka spouts that listen to a topic and at the message receiving, they serialize it as a String and propagate it according to the routing policy. Kafka spouts are implemented using an existing module provided by the official Storm repository. Anyway, as stated also in the code, some modification were made in order to monitor the topology and to fix unexpected behaviours regarding the policy to trigger when an invalid offset is requested (instead of raising an exception, now the spout requests for the earliest offset available for that partition).

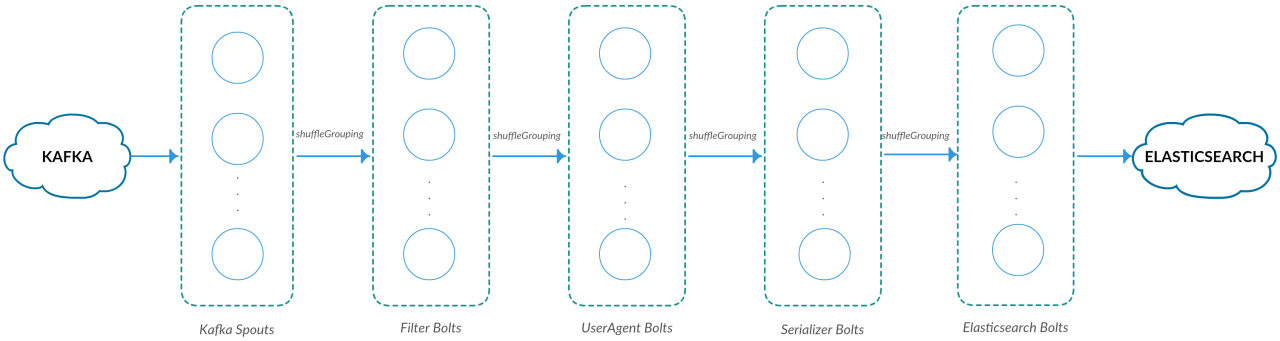


Figure 4.6: Storm real-time topology

Data are then propagated to the next components of the topology, the Filter bolts. The aim of these bolts is just to discard the data we do not need for aggregation. Specifically, all the non-Silverlight events are filtered out. Since into the next step we analyze the user agent of each event, a simple filter function controls the occurrence of that field into the JSON input string received by each Filter bolt. Another option could have been to deserialize the string into a Java object and then check the existence of the user agent field via getter method. Unfortunately, deserializing a JSON string is a time-expensive process and we figured out that checking a substring occurrence is way faster.

The next bolts, as we introduced above, are responsible for the user agent analysis. They break a user agent string into browser and operative system versions which are used for the analysis performed by the serving layer. In this step, an event is deserialized into a *StormEvent* because we need to modify its internal state. After extracting the browser and operative system, these fields are added to the object itself. Another additional field called *counter* is added to the object and its value is initialized to 1. This is needed by Elasticsearch when merging real-time data with batch ones.

After the user agent analysis, data are serialized into JSON strings by the Serializer bolts. Originally this action was executed by the previous bolts but, while tuning

the topology, we discovered a bottleneck in this process and hence it was convenient to separate it from the user agent analysis. Anyway, serializing a *StormEvent* into a JSON String results in a performance improvement when indexing documents on Elasicsearch; moreover, it triggers a denormalization process of the fields used for aggregation, such as the asset name, the video format and the player event type. We will explain the motivation of this choice in the next section.

The final step of the topology is responsible for writing the augmented data into Elasticsearch. As for the Kafka spouts, an already existing implementation provided by Elasticsearch was used and modified for monitoring purposes. In order to improve performance, these components use bulk API for batch indexing. Parameters as the number of documents in a bulk, the bulk dimension in bytes and the flushing time are configurable respectively through *es.storm.bolt.flush.entries.size, es.batch.size.bytes* and *topology.tick.tuple.freq.secs* Storm settings. Part of the tuning process involved trying different values for these parameters in order to maximize the throughput of the topology.

Data are propagated from the components to the next ones in a shuffle mode. As we explained in Chapter 2, this is the best distribution policy when no aggregation processes are executed as it allows to equally spread the load among the various components.

4.5 Serving layer

As stated in Chapter 2, Elasticsearch provides storage space for documents and the *Lucene* search engine for querying them. Moreover, it works with documents in JSON format, making easy to query and aggregate them according to specific fields.

In order to improve performance, Elasticsearch allows to define in advance a mapping model for the documents to be stored, specifying the type of each field (for instance

if a string or an integer) and specific properties such as analyzing policy (make a field available for a full-text search) or whether to index that field or not. Setting such a mapping is very important and normally fields which will not be used should not be indexed at all or made not available for advanced search features.

The mapping used in this implementation is straightforward, each fields is mapped with its natural type (text values mapped with strings, numeric values with integer or long) and all the text values are mapped as *not_analyzed*; hence, we do not need full-text search. In order to merge real-time and batch data, we need to perform operation on the same field, this is the reason of the data denormalization process executed in the real-time layer. As explained in Section 4.3, results produced by the batch layer contain a field for the subject being aggregated and its value. That field name must match the corresponding one inside the real-time event so that it is possible to sum the *counter* values into a unique result.

Two Elasticsearch indices were defined:

- *vimond-realtime*
- *vimond-batch*

and each layer stores its output in the corrisponding type.

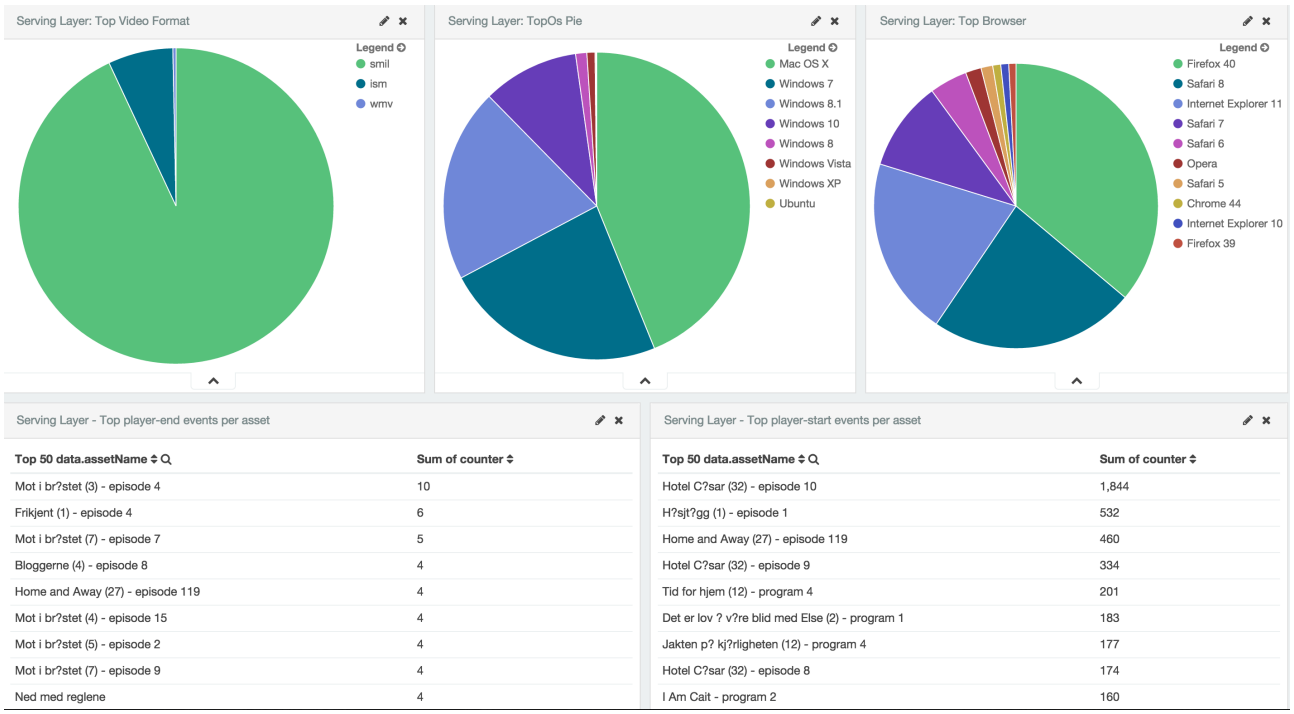


Figure 4.7: Kibana dashboard

Elasticsearch itself provides REST or Java API for making data aggregation or querying. In order to have a user interface, we used *Kibana*[2], a visualization tool included into the Elasticsearch products suite that uses REST API for visualizing and executing operations on data stored in the cluster. Figure 4.7 shows part of the VimondAnalytics Kibana dashboard.

Under the hood, Kibana sends GET requests with a payloads as the one reported in listing 8. Both types are queried and the aggregation is executed just on the query result. In the listing, data are filtered using the *query* field from the request, which returns all the documents from *vimond-batch* having *eventName* equal to *"StartEvent-PerAsset"* and all the ones from *vimond-realtime* having the field *data.playerEvent* equal to *"str-start"*. Elasticsearch defines two families of aggregation: bucketing and metrics. The first just divides data into buckets according to some criterion, while the

[2]http://www.elastic.co/products/kibana

metrics actually aggregates data inside each bucket. In the example related to Listing 8, data are first divided into buckets according to *data.assetName* and then aggregated by *counter*. Data from real-time layer have the counter set to 1, while the ones from the batch layer to the aggregated value. Summing these fields values actually merges the two types of data.

```
"size": 0,
"aggs": {
  "2": {
    "terms": {
      "field": "data.assetName",
      "size": 50,
      "order": {
        "1": "desc"
      }
    },
    "aggs": {
      "1": {
        "sum": {
          "field": "counter"
        }
      }
    }
  }
},
"query": {
  "filtered": {
    "query": {
      "query_string": {
        "analyze_wildcard": true,
        "query": "eventName:\"StartEventPerAsset\"
         || data.playerEvent:\"str-start\""
      }
    },
    "filter": {
      "bool": {
        "must": [
          {
            "range": {
              "timestamp": {
                "gte": 1379686790025,
                "lte": 1442758790025
```

```
                }
              }
            }
          ],
          "must_not": []
        }
      }
    }
  }
```

Listing 8: Start events per asset JSON request

4.6 AWS Deploy

We have deployed the VimondAnalytics system on a private VPC (virtual private cloud) provided by Amazon Web Services (AWS). We chose AWS since Vimond already uses it for its infrastructures. Moreover, the company providing input data (TV2) runs as well its environments there. Anyway, it is possible to deploy our system on any cloud provider as it does not use exclusive services of Amazon, but just basic processing and networking resources. A related example is provided by the Hadoop deployment choice: Amazon provides a service called Elastic Map Reduce[3] that allows to fire up a working Hadoop cluster (including HDFS) hiding all the complexity of tuning and setting it. However, in our system de decided to deploy the Hadoop cluster on separate virtual machines and part of the project effort was to manually tune the settings in order to achieve the best performances for the scenario we considered.
We report below the configuration of the virtual machines used for each layer of VimondAnalytics.

- Data ingestion: *t2.medium* (2 vCPU Intel Xeon up to 3.3 Ghz, 4 Gb RAM, 8 Gb EBS).

[3]https://aws.amazon.com/elasticmapreduce

- EventFetcher: *t2.small* (1 vCPU Intel Xeon up to 3.3 Ghz, 2 Gb RAM, 8 Gb EBS).
- Processing layer:
  - Master node: *m4.large* (2 vCPU Intel Xeon E5-2676 v3 2,4 Ghz, 8 Gb RAM, 8 Gb EBS)
  - Slave node *m4.xlarge* (4 vCPU Intel Xeon E5-2676 v3 2,4 Ghz, 16 Gb RAM, 60 Gb EBS)
- Serving layer: *m4.xlarge* (4 vCPU Intel Xeon E5-2676 v3 2,4 Ghz, 16 Gb RAM, 30 Gb EBS)

The virtual machine sizes choice was driven by their scopes. Processing and serving layers virtual machines deal with a bigger workload than the others, hence they require a more powerful virtual hardware. We empirically checked that, instances less powerful than *m4.xlarge*, resulted in a bottleneck in the CPU in the processing and serving layers. The following services were deployed on each specific machine:

- Data ingestion: Kafka broker, Zookeeper, Exhibitor (Zookeeper monitoring tool);
- Event fetcher: one instance of event-fetcher program;
- Master node: HDFS Namenode, YARN Resource manager, Storm Nimbus, Oozie;
- Slave node: HDFS Datanode, YARN Node manager, Storm Supervisor;
- Serving Layer: Elasticsearch.

The default configuration used for a typical execution of the architecture is:

- 1 instance of Data ingestion;
- 1 instance of EventFetcher;
- 1 instance of Master node;
- 2 instance of Slave node;
- 1 instance of Serving layer.

We opted for this configuration in order to minimize the overall cost of the infrastructure altough it is not the optmimal one in terms of scalability and fault tolerance. Figure 4.8 shows the machine diagram for the default configuration.

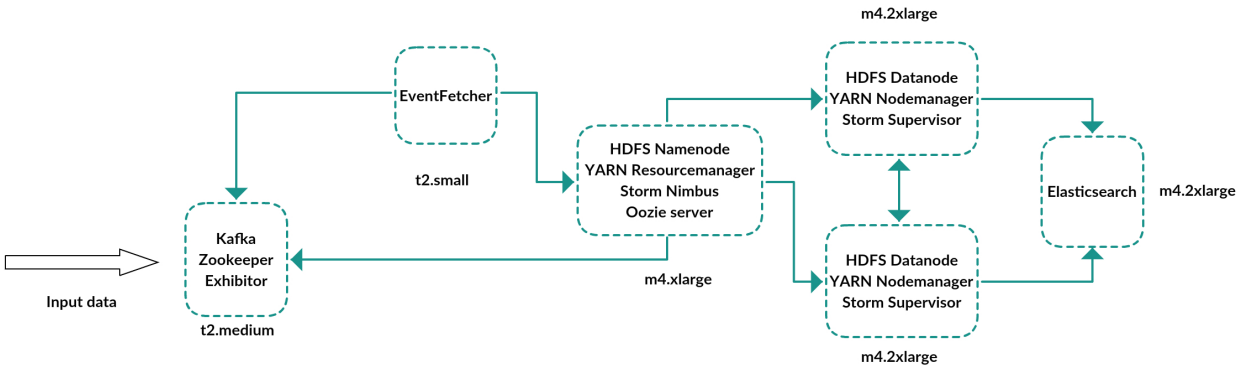


Figure 4.8: AWS default configuration

In order to minimize the number of used virtual machines, the batch and real-time processing layers share the same ones. By properly setting their configuration files, resource starvation problems can be avoided. Spark was not directly deployed on any machine, but its executable libraries must be copied on HDFS in order to be shared among the slave nodes. When a batch job is scheduled, each slave node fetches these libraries in order to execute the job itself.

Every deployed service is executed in supervised mode using *daemontools*[4] commands

[4]http://cr.yp.to/daemontools.html

suite. This implies each process to start at machine boot time and restart in case of failure of the process itself. Some components of the frameworks used in this implementation are *fail-safe* and require to be executed in supervised mode, which means that in case of their failure, they can be restarted without affecting other parts of the system as they will restore their state from a safe checkpoint.

Chapter 5

Experimental results

In this chapter we will show the results of the experiments run on a working implementation of VimondAnalytics. First, we will describe how we measured the system KPIs for each layer. Then we will show the experiments methodologies and their results. We executed two types of test aiming at measuring the efficency of the Lambda architecture, the robustness and the fault-tolerance of the system in response to failover scenarios for each different layer.

5.1 Monitoring the system

In order to monitor the system and get statistics about KPIs (key performance indiators) of each layer such as throughput and latency, we have used the library *dropwizard-metrics* [1] (formerly *codahale metrics*). This library allows to define different types of metrics for counting the ratio of a certain event or the execution time of a function. Each metric refers to a *MetricRegistry* object which accumulates all the received data and publishes them using one or more *Reporter*. Example of supported reporters are *CSVReporter*, *HTTPReporter* and *GraphiteReporter*. For our purposes we used a *CSVReporter* which appends the report for a configurable time window to CSV file.

[1] http://dropwizard.github.io/metrics

5.1.1 Real-time Layer

For the Storm topology we are interested in two metrics:

- throughput of each component;
- latency of a tuple, i.e. the time each message spends into the topology, from the moment it is emitted by the spout till the moment it is processed by the last bolt.

The throughput of each component is measured as the number of tuples executed by that component (either a spout or a bolt) in a fixed time span, for instance a second. The latency, is measured as the difference between the time when a tuple is processed by an Elasticsearch bolt and the time when that tuple is created by a Kafka spout. The latter value is emitted together with the message itself and propagated with it through the whole topology. This approach relies on having the system clock of each node in the cluster synchronized with the others or with an external trustful source. The easiest way to achieve that is to synchronize the system clock of each server with an external source using the NTP protocol. Under Linux, this can be done by installing the *ntp* package which runs a *ntpd* daemon synchronizing the system clock with a NTP server at startup time and at every configurable interval. Each server must have access to the public Internet and, on AWS, this can be done using an elastic IP or configuring the instance to have a public IP at creation time.

5.1.2 Batch Layer

In the EventFetcher process we are interested in the throughput of each instance. Since it is responsible for fetching messages from Kafka e write them on HDFS, the throughput comprises both actions.

Concerning the Spark module, we keep track of the execution time of each aggregation process. The code is designed such that the main function waits the computation to be over before exiting the program. Using a *Timer* object from the monitoring library allows to calculate the exact running time. The same has been done for the function which deletes real-time records after completing a batch process.

5.1.3 Serving Layer

In this layer we are interested in two metrics:

- Number of ndexed documents;
- Average query time.

The first metric is trivial as it is just the result of a query which matches all the documents stored in Elasticsearch. The average query time, is the result of the average of the different times needed for executing each example query defined in Kibana. Elasticsearch provides Java API for interacting with a cluster, hence the same example queries have been translated in order to access the serving layer from outside Kibana.

5.2 Testing VimondAnalytics

In this section we will show the results of the experiments we run on a working implementation of VimondAnalytics running on AWS. When not mentioned, the settings and the virtual machine configurations are the same as indicated in Section 4.6.

5.2.1 Experimental methodologies

The data ingested in real-time from the TV2 environment do not fit very well for a test scenario for two reasons: first, the frequency of data ingestion is not constant and

overall the throughput is way to low. Secondly, it is not possible to replicate the same scenario for several executions of the same test or similar ones. Using directly the production data would make a test not consistent with the others.

To solve this problem, we have designed a safe test environment context, leveraging the log retention property of Kafka. Another broker is replicated in the same VPC where the system normally works, just with the purpose of accumulating enough data for running the tests. Data are propagated into a topic in the new Kafka instance and saved there using a higher log retention policy. For running the tests, the same data are written into another topic of the same instance with a throttled rate and both the Storm topology and the EventFetcher process read data from that topic. Using this approach enables to decide the data input frequency and to run several tests on the same dataset. Figure 5.1 summaries the test environment.

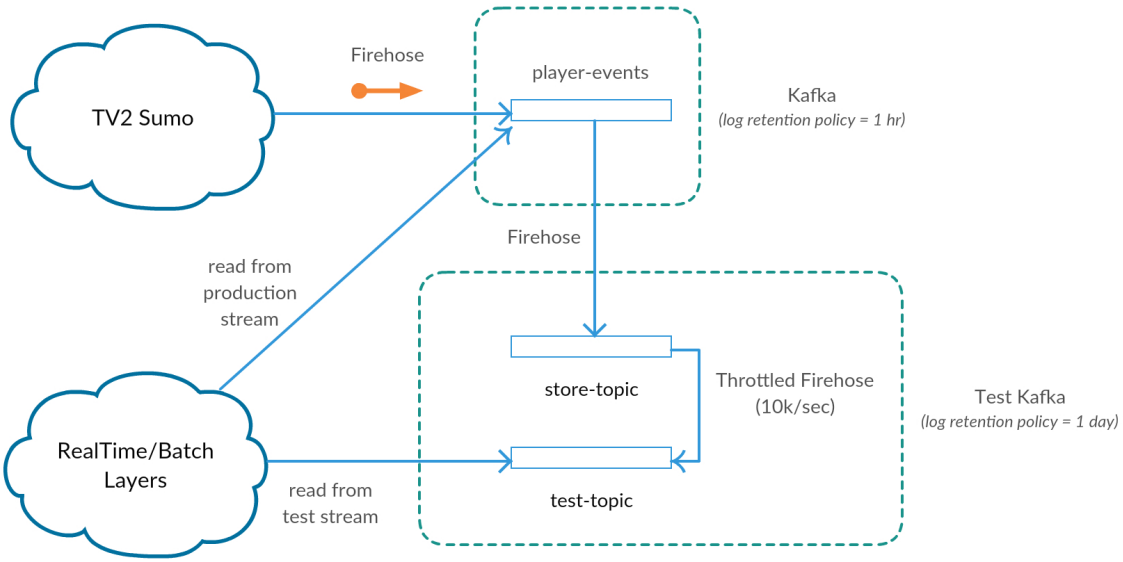


Figure 5.1: Test environment: input data are read at a throttled rate

5.3 Tests on the efficiency of the Lambda architecture

The first test we executed on the system acts as a proof of concept of the Lambda architecture approach. The goal is to analyze the difference between having only realtime and serving layers versus a complete system, in terms of number of documents indexed into the serving layer and average query time. Both the executions use the same dataset injected at a frequency of 10K messages per second, according to the methodology explained in Section 5.2.1.

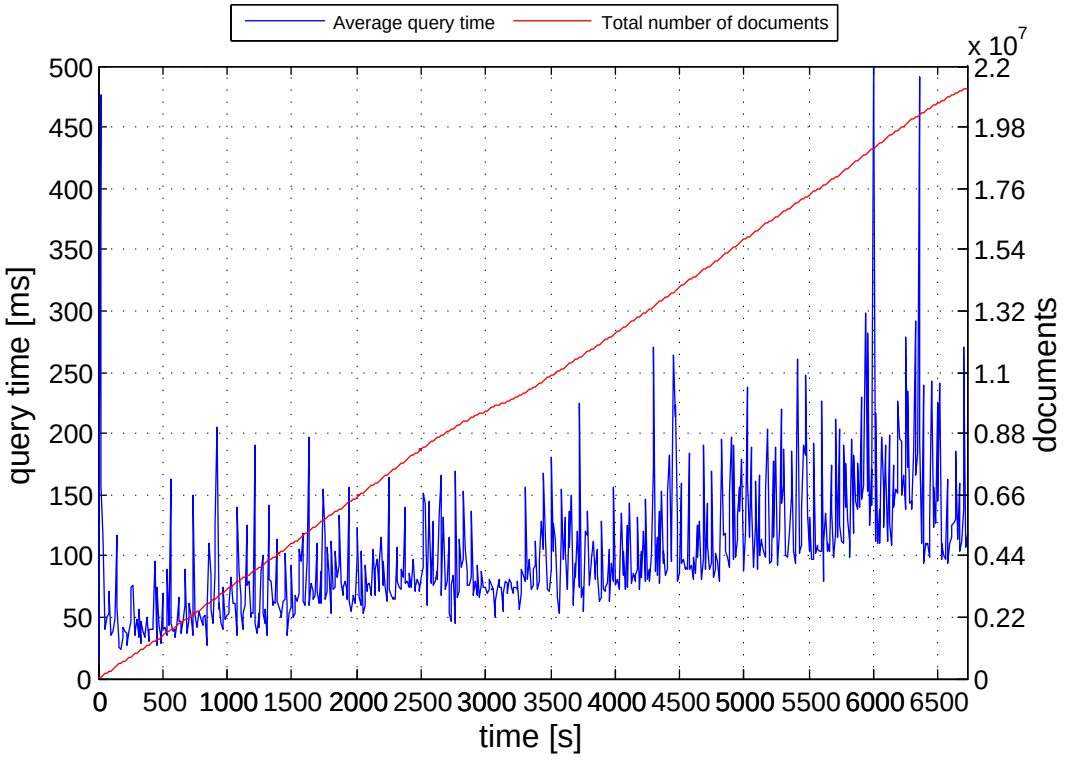


Figure 5.2: RealTime layer only: average query time vs number of documents

A dataset of 45 Gb was processed by both systems. In the real-time-only one, data

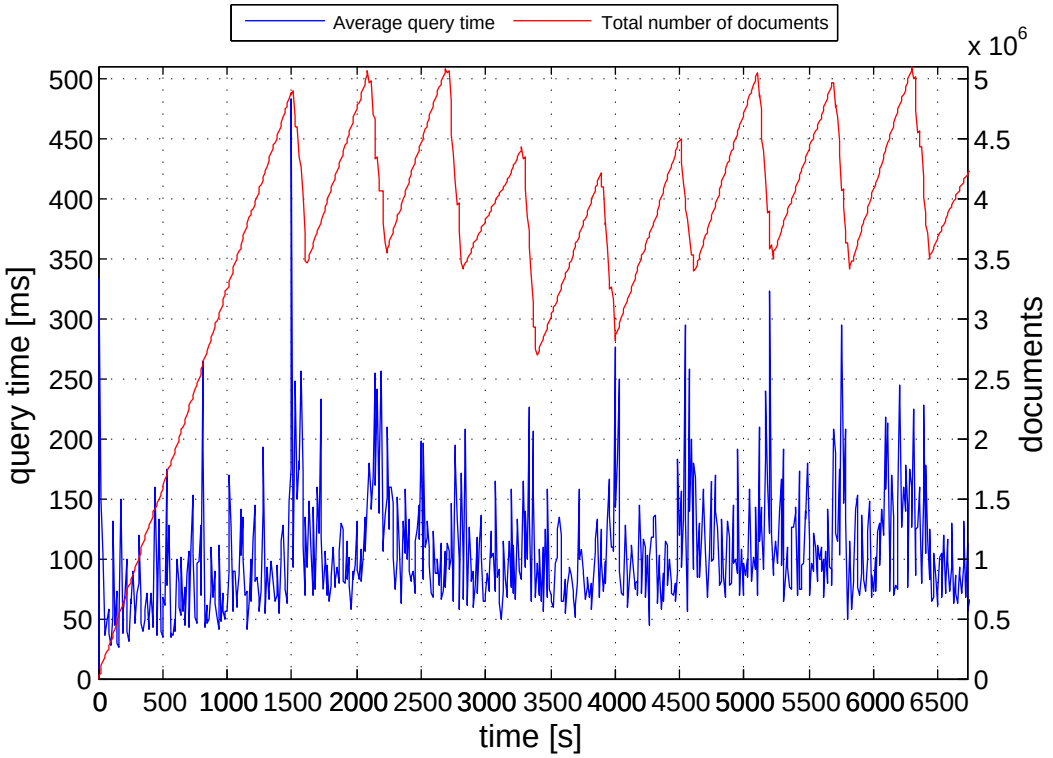


Figure 5.3: VimondAnalytics: average query time vs number of documents

are ingested into the serving layer which aggregates them for satisfying the queries requests. As illustrated in Figure 5.3, the number of indexed documents grows with the time, since there is no deletion mechanism, and after 1:45 hours of execution, there are more than 20 million documents stored. Even though Elasticsearch can scale out very well even with one instance, the average query time grows as well according to the number of documents to query. It goes from ∼50 ms to ∼200 ms and it keeps growing all over the time. We can conclude that, the real-time-only layer requires the serving layer to scale out to more instances as the number of documents reaches a threshold which causes a substantial increase of the query time.

VimondAnalytics avoids this problem by deleting the real-time events when a corresponding batch job is successfully executed. It results in a number of documents equals to the one ingested by the real-time layer in the time frame not covered by batch jobs, plus a small amount representing the aggregated data for the previous time windows. The query time is also influenced by this documents handling policy since, rather than of always increasing, it reaches local maxima in corrispondence of a start of a deletion event. The spikes of its trend are due to the configuration used for this test: an instance of Elasticsearch has to satisfy at the same time requests for batch insertion and deletion; hence, the corresponding query time is hence affected. Apart to these temporal behaviours, the average query time keeps growing till the first deletion event, then it stays inside the interval [50,150] ms.
The next tests refer to the same dataset processed by VimondAnalytics implementation with different batch time windows. The cases analyzed are time windows of 5, 10 and 15 minutes. The following configuration was used for each test case:

- Data injection: one instance as described in Section 4.6
- RealTime layer: three instances as described in Section 4.6 with
  - Kafka spout: degree of parallelism of 2
  - Filter bolt: degree of parallelism of 2
  - UserAgent bolt: degree of parallelism of 2
  - Serializer bolt: degree of parallelism of 2
  - Elasticsearch bolt: degree of parallelism of 3
- Batch layer:
  - EventFetcher: one instance as described in Section 4.6

  - Yarn: three instances as described in Section 4.6

- Serving layer: one instance as described in Section 4.6.

The number of machines used for these tests was chosen according to what stated in Section 4.6. Since we are not testing failover scenarios, we are not interested in components replication. We checked empirically the Storm topology degree of parallelism of each task in order to avoid any bottleneck in the computation. We set the higest replication factor for the Elasticsearch bolt since it resulted to be the slowest process.

5 minutes batch window

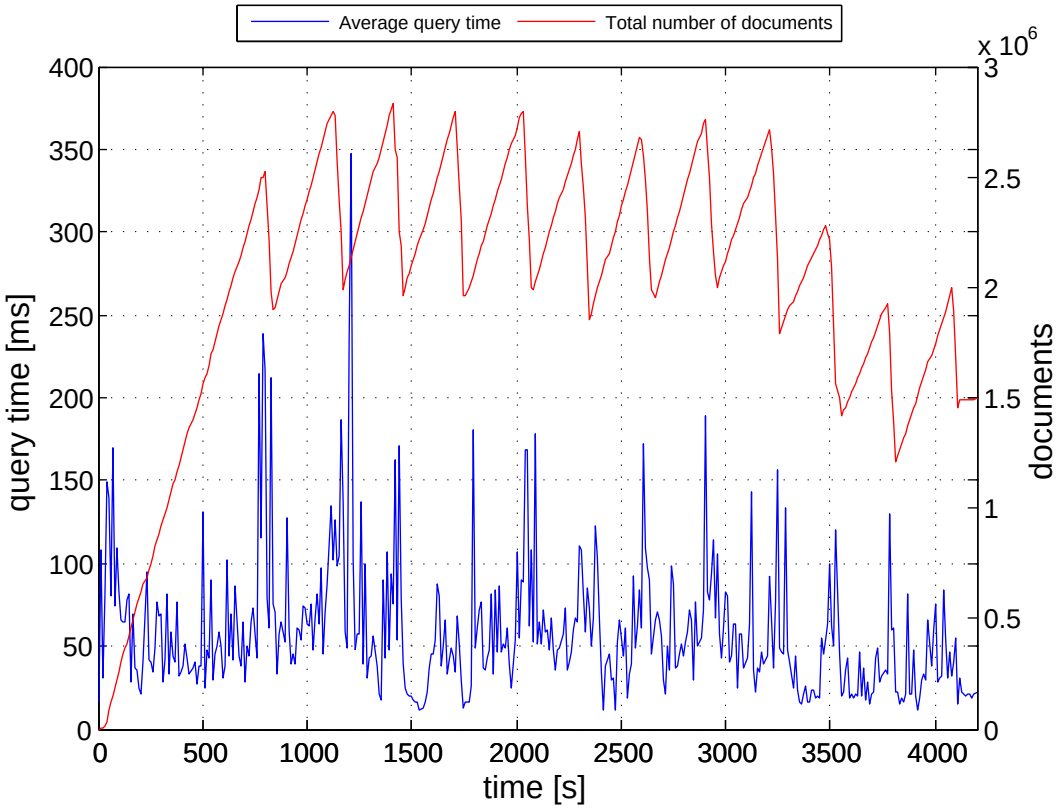
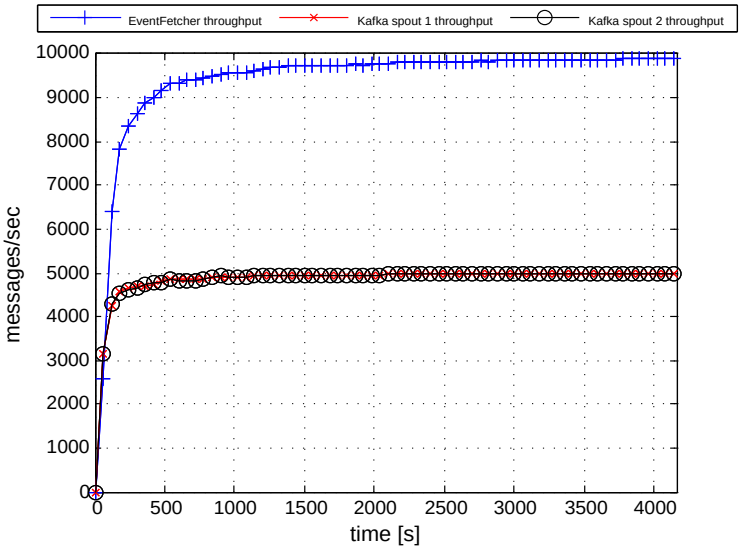
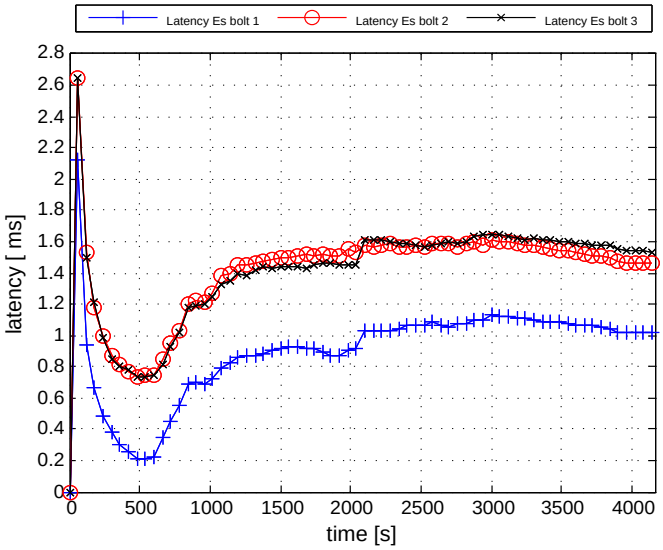


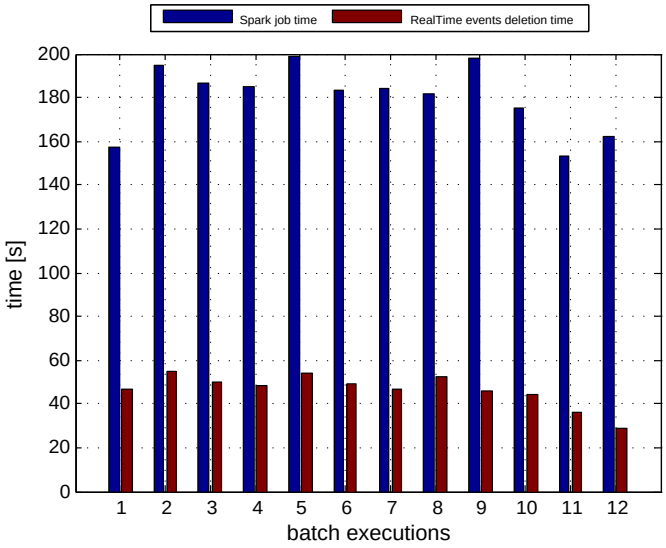
Figure 5.4: VimondAnalytics: batch window of 5 minutes



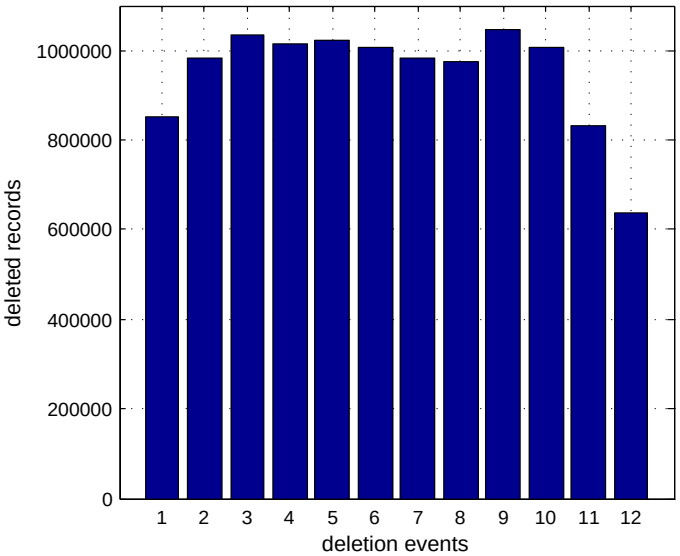
(a) Input throughputs



(b) RealTime layer latency



(c) Jobs execution time



(d) RealTime events deleted for each batch execution

Figure 5.5: VimondAnalytics metrics for a batch window of 5 minutes

10 minutes batch window

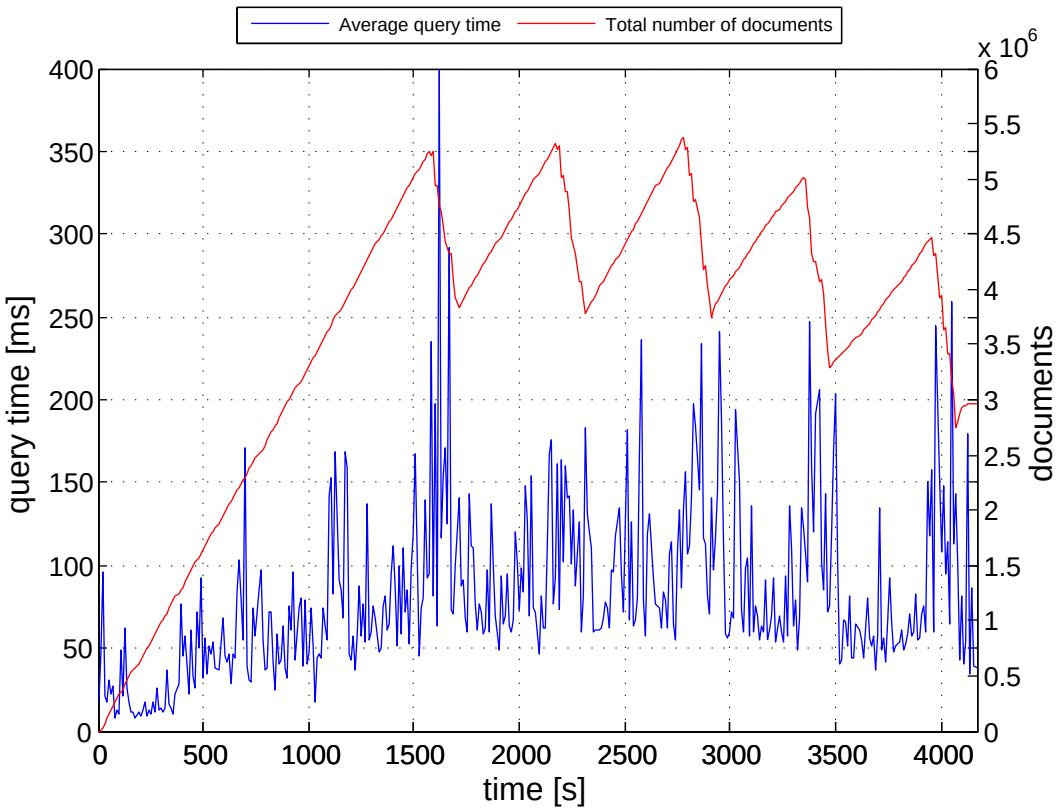
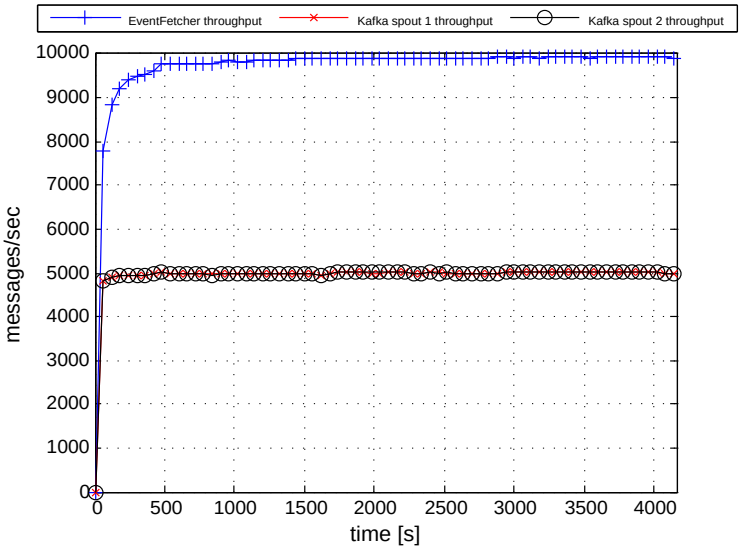
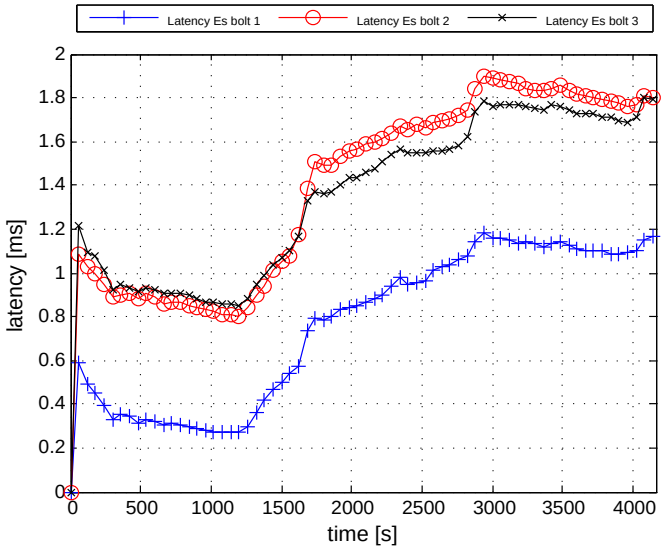


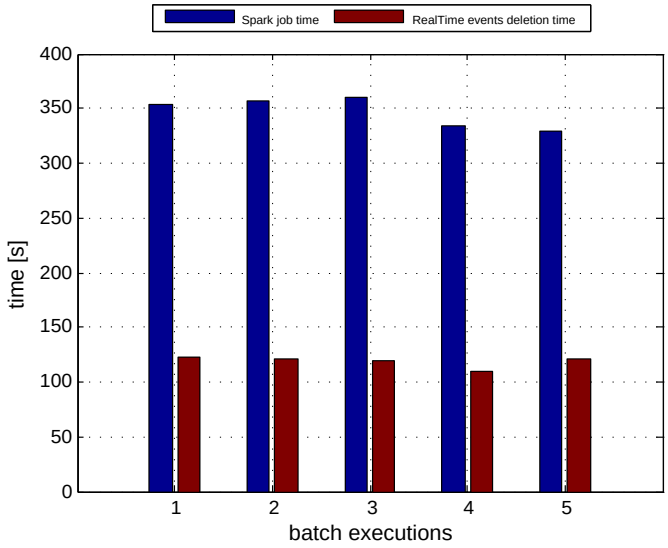
Figure 5.6: VimondAnalytics: batch window of 10 minutes



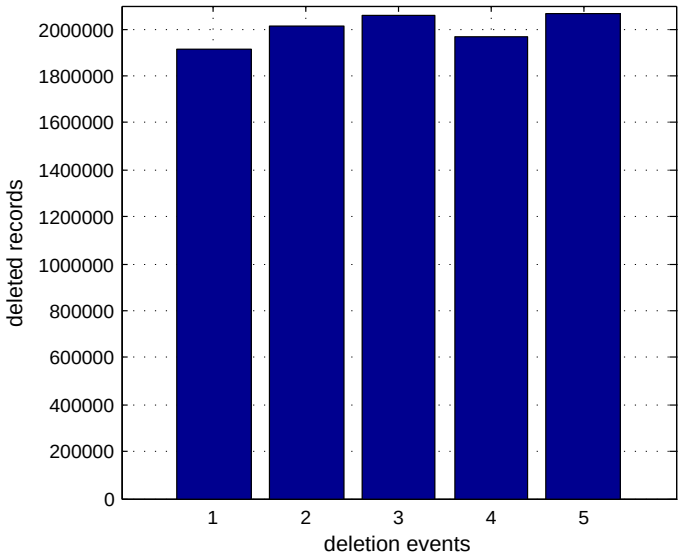
(a) Input throughputs



(b) RealTime layer latency



(c) Jobs execution time



(d) RealTime events deleted for each batch execution

Figure 5.7: VimondAnalytics metrics for a batch window of 10 minutes

15 minutes batch window

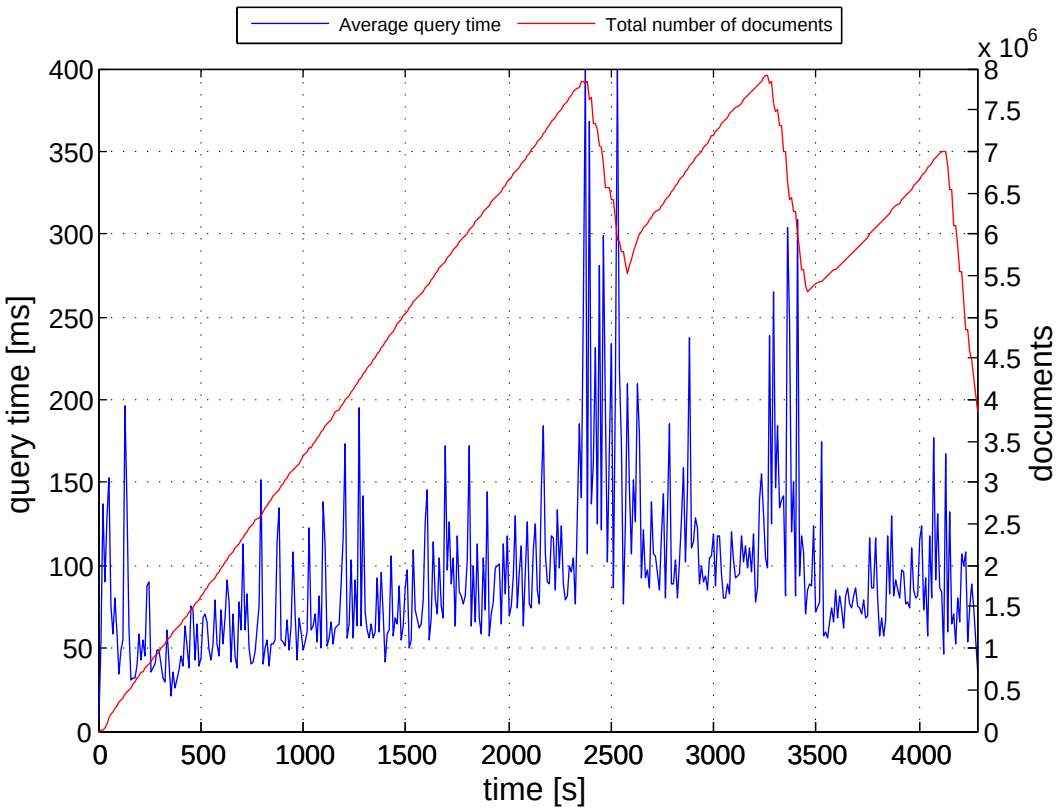
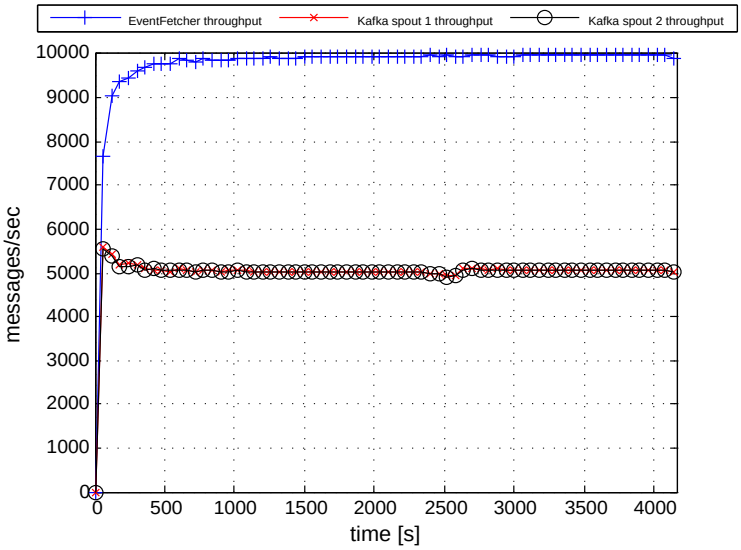
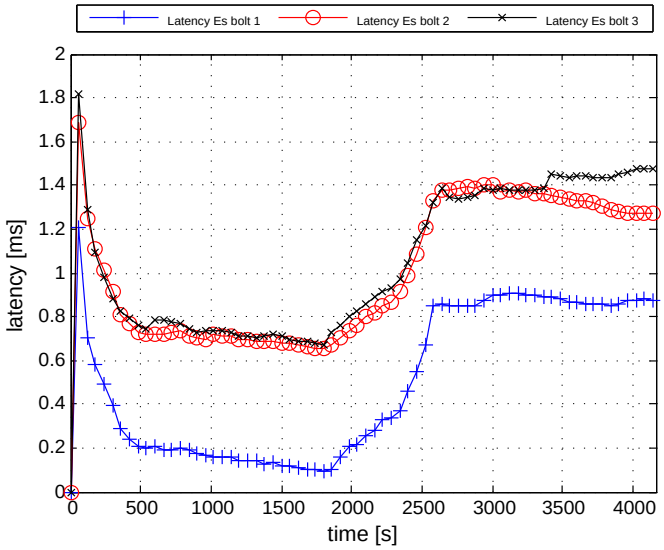


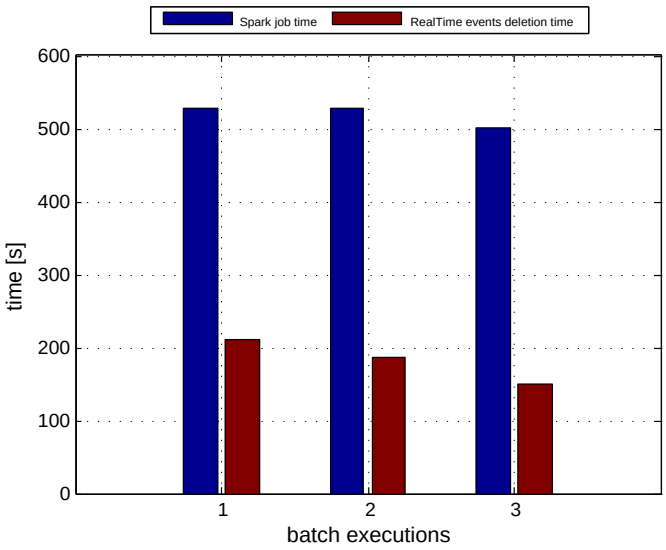
Figure 5.8: VimondAnalytics: batch window of 15 minutes



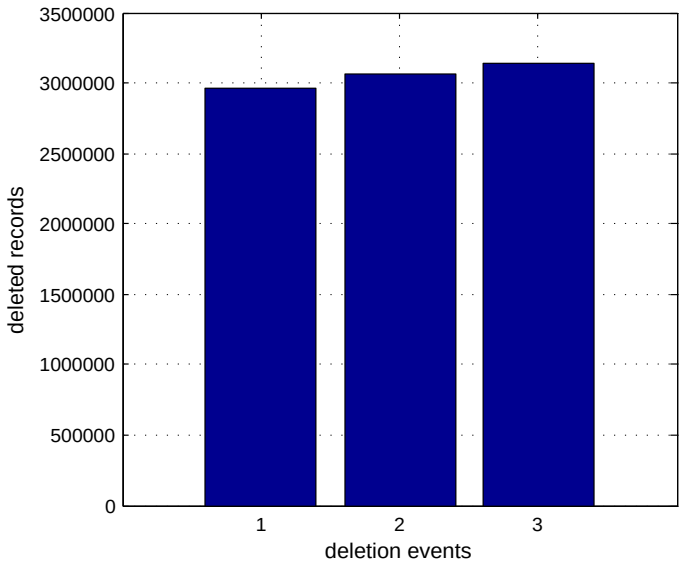
(a) Input throughputs



(b) RealTime layer latency



(c) Jobs execution time



(d) RealTime events deleted for each batch execution

Figure 5.9: VimondAnalytics metrics for a batch window of 15 minutes

Analysis of the batch window results

The previous tests referred to a same dataset of 27 Gb analyzed by VimondAnalytics implementation in the cases of batch timewindows of 5, 10 and 15 minutes. In all the three query time trends we can distinguish the same pattern (Figures 5.4, 5.6 and 5.8). At first, the number of documents indexed into the serving layer starts growing as does the corresponding query time. With the first deletion event, we have the first spike in the query time, which then decreases and increases again along with the executions of the next deletion events. The only difference between the three different tests lies in the values that the number of documents and the query time assume.
For each test execution, we reported metrics regarding the behaviour of each component of the system, as showed in Figures 5.5, 5.7 and 5.9. Apart from the throughputs of the input components, which have the same trends in every execution due to the throttled input rate, they assume different values influenced by the current time windows. The most interesting one is the latency of the real-time layer. After an initial spike, the latencies of the Elasticsearch bolts tend to stabilize to values comprised between 0.2 and 0.8 ms. They stay stable till the first deletion event, which cause a rapid increase and after that, the final bolts of the Storm topology adapt their execution with Elasticsearch and eventually the latencies converge to values from 0.8 to 1.8 ms. This behaviour is clearly identifiable in all the executions, with the only difference of the time at which the first deletion event occurs, respectively after 10, 20, and 30 minutes.

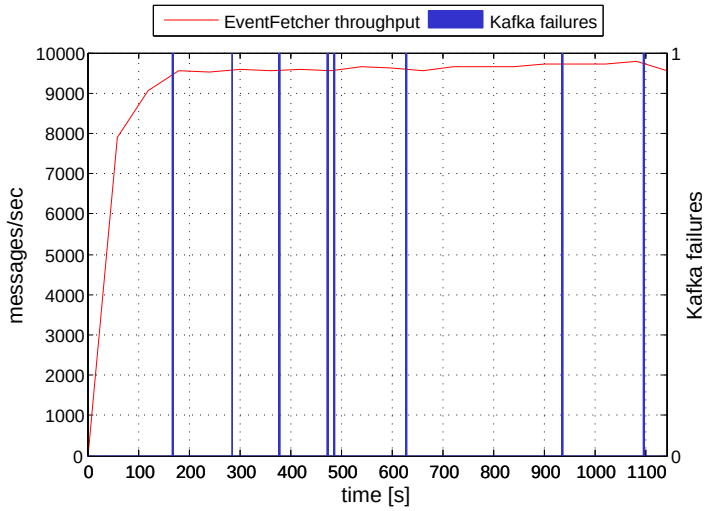
5.4 Failover Tests

In this section we will show the results of experiments made on some failover scenarios. We will refer to a kill action when a *SIGKILL* signal is sent to the process (*kill -9* unix

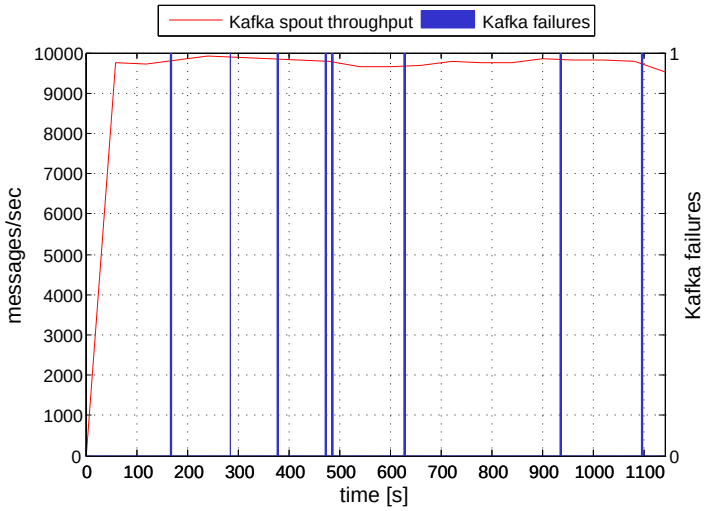
command), while a stop/start action represents a kill action followed by a restart of the process.

5.4.1 Kafka failover

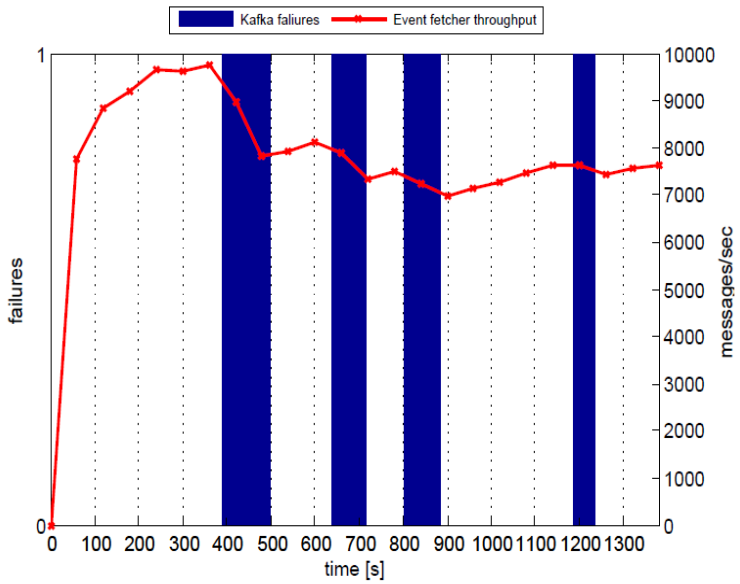
The first failover test investigates the reaction of the input components of VimondAnalytics (Kafka spout and EventFetcher processes) when there is a failure in the data ingestion part. We simulated two type of failures in the Kafka broker: a kill of the process and a stop/start action. Since also Kafka is launched in supervised mode, we expect that killing the process does not cause any delay or message loss in the system. As expected, killing the Kafka process does not influence the throughput of any input component, as after a failure, they just keep waiting for its restart, as shown in Figures 5.10 (a) and (b). Figures 5.10 (c) and (d) show what happens when Kafka is stopped and restarted after a random time. We can notice a drop in the throughput, both in the Kafka spout and in the EventFetcher process. This is absolutely normal since, for the time Kafka is inactive, messages are not ingested into the system. Finally, when the broker is restarted, each component increases its throughput due to the higher data input frequency. Each stop/start event causes this pattern in the throughput trends, and results in a higher processing time respect to the case when Kafka is just killed and immediately restarted. The number of read messages is the same in both the test scenarios.



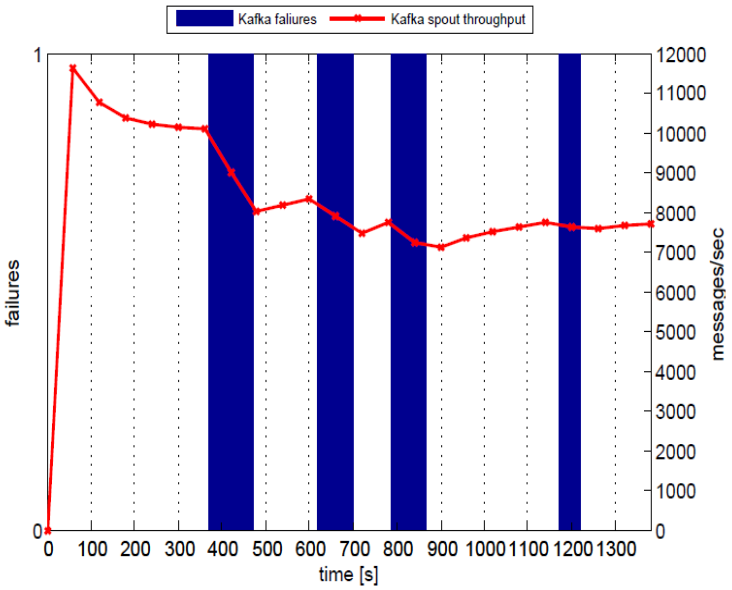
(a) EventFetcher throughput with Kafka kill actions



(b) Kafka spout throughput with Kafka kill actions



(c) EventFetcher throughput with Kafka start/stop actions

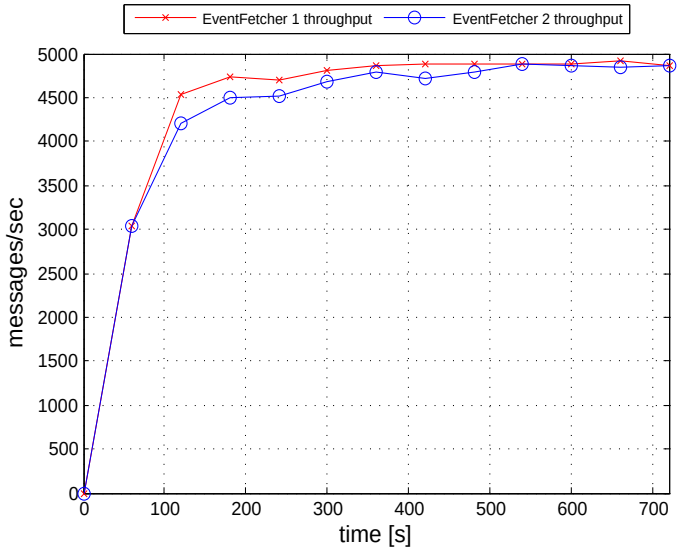


(d) Kafka spout throughput with Kafka start/stop actions

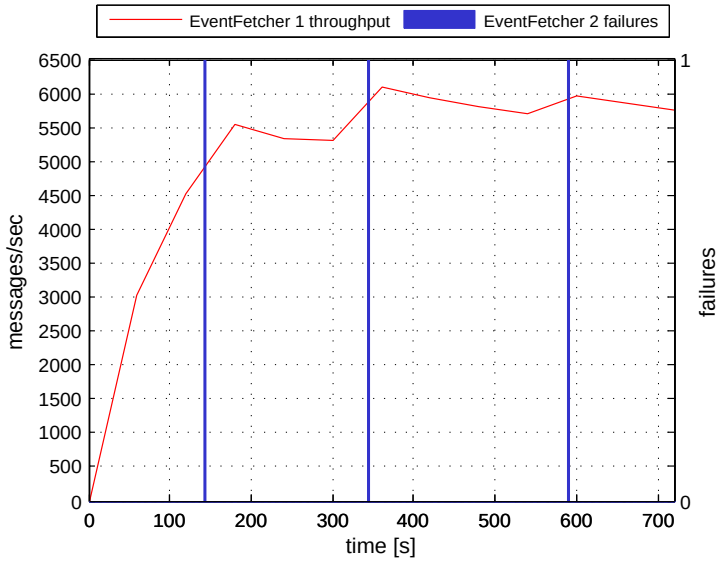
Figure 5.10: Input throughputs variation for a Kafka kill and stop/start action

5.4.2 EventFetcher failover

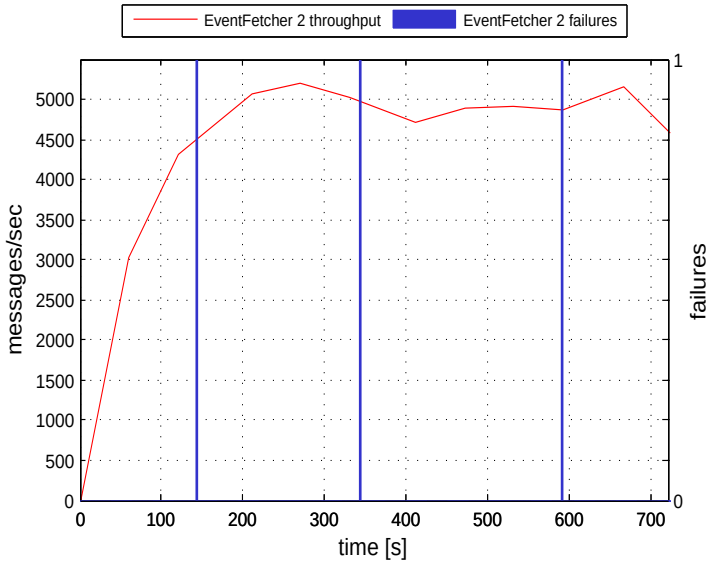
The next set of experiments aims at investigating the fault-tolerance of the EventFetcher process in case of its failures. Two instances of this process are used, one operating normally and the other having failures. As for the previous test, we simulated a process shutdown by sending it a *SIGKILL* signal and a stop/start action.
In case of no failures, we can notice that the instances divide equally the input messages by converging to the same throughput (Figure 5.11 (a)). We recall that the throughput is slightly less than the input frequency due to the time needed for writing messages on HDFS. In case of several kill events, we can notice a rapid increase of the throughput of the healthy instance, caused by the rebalancing mechanism of Kafka within a consumer group. Indeed, for the time the second instance is not active, all the topic partitions are assigned to the healthy one. Since also the EventFetcher runs in supervised mode, it is restarted immediately after being killed, and consequently the throughput will increase or decrease in order to converge again to the same value. Figures 5.11 (b) and (c) show the throughput trends for the healthy process and for the one being killed.
In case of stop/start actions, we can recognise the same trend of the previous case, with the throughput of the healthy instance that increases and decreases according to the functioning of the unhealthy instance. The difference is how fast the rate changes: during the first stop action, it reaches almost 9k messages/second, and later it drops when both the processes are working correctly. The same happens during the second stop action, as we can notice in Figure 5.11 (d).



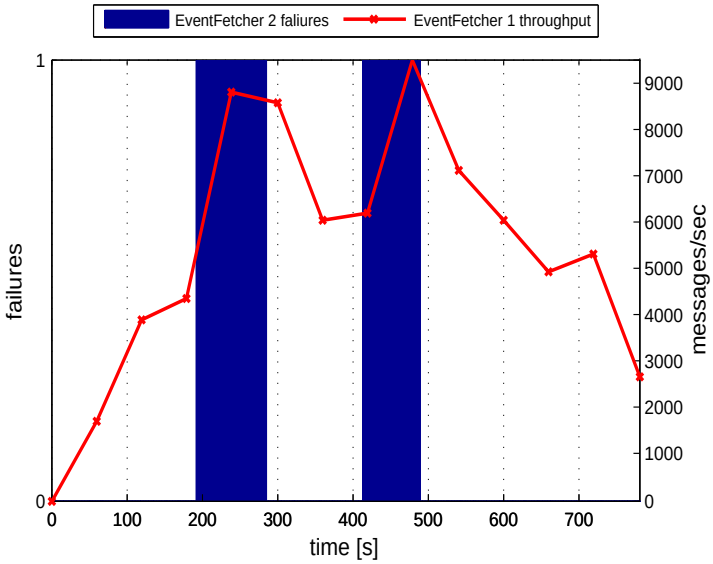
(a) EventFetcher throughput with no failures



(b) EventFetcher 1 throughput with kill actions



(c) EventFetcher 2 throughput with kill actions



(d) EventFetcher 1 throughput with stop/start actions

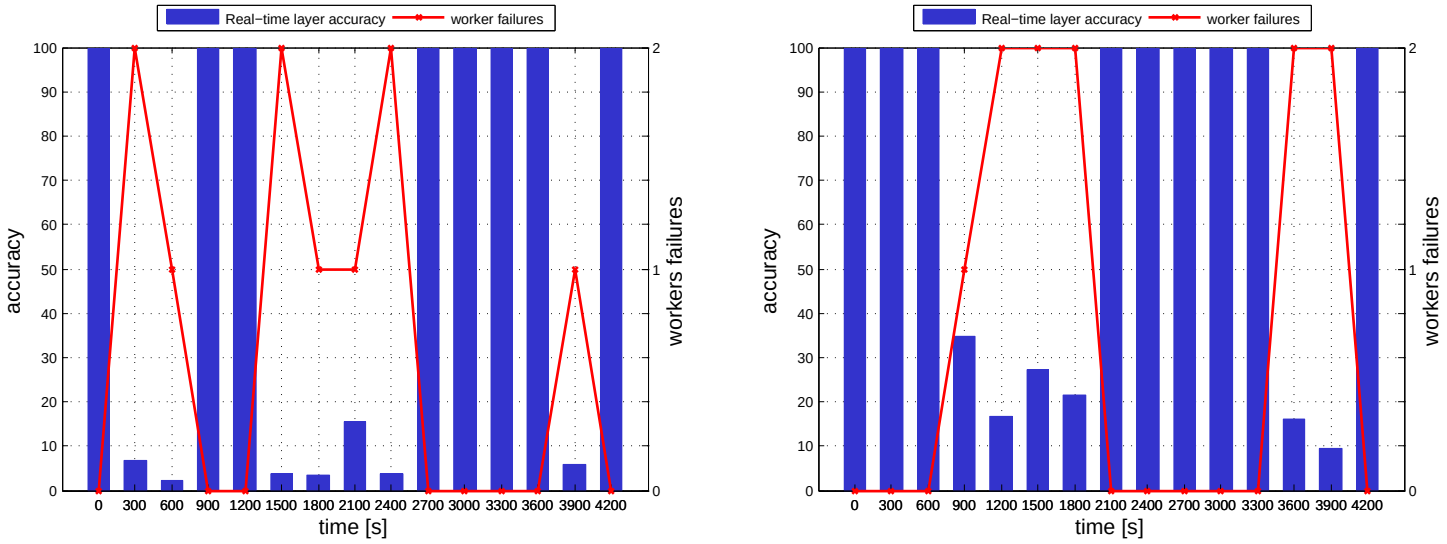
Figure 5.11: Input throughput variation for EventFetcher kill and stop/start actions

5.4.3 Storm failover

With the third failover test we want to show how the accuracy of the results produced by the real-time layer is affected by Storm failover events. The accuracy is measured with respect to the results of the batch layer for the same time window of data. In order to simulate a failure only in the real-time layer we can randomly kill *worker* processes or the *supervisor* for a node among the slave ones. Killing the *supervisor* does not affect the normal processing as it is involved just in coordination and monitoring operations. Moreover, it is a *fail-safe* process, hence, after being automatically restarted, it comes back to its normal functioning.

Killing a *worker* process is more interesting. It results in one or more killed executors and then in a consequent message loss. Once a supervisor process noticed a failure in a worker which is under its control, it tries to restart it on the same machine. In case of continuous failures, *Nimbus* schedules the execution of that worker on another machine of the cluster. As stated before, Storm allows to run a topology in reliable mode exploiting the acking mechanism, so both scenarios were tested.

In case of acking topology, we experiment message duplicates, due to the *at-least-one* message semantic offered by Storm. Even though there are drops in the accuracy (Figure 5.12 (a)), there is no message loss. In our implementation context, this does not change the final result, as the serving layer is agnostic to possible message duplicates. Elasticsearch provides a method for executing aggregations based on a unique message field value, but as for most of databases, it uses the *HyperLogLog* approximation algorithm. Anyway, avoiding message loss could be useful in other implementations with a serving layer aware of messages duplication. In case of unreliable topology, killing worker process leads to message loss and consequently to a drop in the accuracy as shown in Figure 5.12 (b). For our test queries, we notice that, loosing messages im-



(a) Real-time layer accuracy with acking

(b) Real-time layer accuracy without acking

Figure 5.12: Storm failover: variation of the accuracy of RealTime layer w/o acking in presence of worker failure

plies having a slightly higher accuracy respect to the reliable execution mode. This is anyway, totaly dependant on the test queries and might not be true for other ones.

Both these tests prove one of the key concept of the Lambda architecture, and therefore of the proposed approach, which is the eventual consistency. Drops in the accuracy of the real-time layer, either for component failures or just for inaccuracy of the results due to approximation methodologies, are eventually corrected by the batch layer. Specifically, in VimondAnalytics model, the results are consistent with a time frame after $2 * batch\_time\_windows + \delta$, where δ is processing time of a batch of data (aggregation plus real-time events deletion time).

5.4.4 Yarn failover

In the last failover test, the same dataset of 27 Gb is analyzed by the Yarn/Spark module twice, the first time without errors, while the second time a random Yarn

container is killed during each execution. The processing time of each Spark process was measured according to what explained in Section 5.1. A 5 minutes time window is used, but similar results can be obtained for different values.

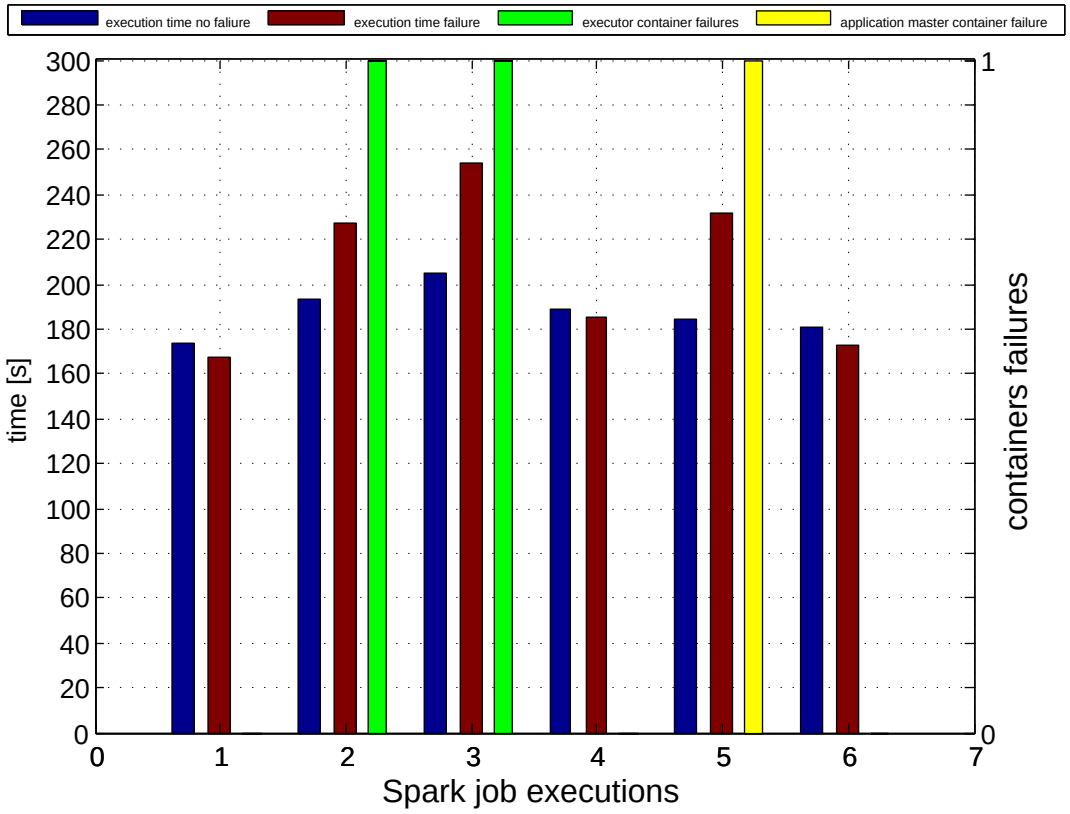


Figure 5.13: Yarn failover: variation of batch job execution time

In order to understand the results of this test, we have to keep in mind how Yarn distributes the execution of a Spark job in several containers and how it handles their possible failures. As explained in Section 4.3, Spark is executed in *cluster-mode* on Yarn, which causes the *driver program* to run in a container along with the Yarn *application master*. Since we have two executors in the current implementation, each Spark job consists in three containers: one for the *driver/app. master* and one for each

executor.

A failure in an executor container causes the *application master* to request *resource manager* to allocate a new container with a new executor. Leveraging the distribute nature of Spark *resilient dataset*, the new executor computes its functions only on the partitions it was responsible of. The job execution is not cancelled, but its processing time results in higher values due to the executor restart time. By default, after two executor container failures in the same application, the overall job is considered failed. In Figure 5.13 we have an executor container failure in the executions 2 and 3. As a consequence, the processing time results increased by a value depending on when the container fails. The later it happens, the longer the execution time as the new executor has to calculate more steps for its dataset partitions while the other one stays idle waiting it to finish the recovering process.

On the other hand, a failure in the *application master* container is more dangerous, as it causes a failure in the current application. In this situation, the *resource manager* kills every active container for that application and starts a new attempt of it. By default, after failing two attempts, the overall job is considered failed. In our test, there is an *application master* failure in the 5th execution. As a result, the overall processing time is increased by the time at when there was a failure, since a new application execution is started after it.

5.5 Tests results

In this chapter we analyzed the performance of the system realized in this thesis project. We showed how the proposed model beats a classic approach consisting in storing documents and aggregating them on-the-fly. VimondAnalytics is superior in terms of number of stored documents and, consequently, query time. The drawback of this ap-

proach consists in the spikes of the query time in correspondence of the deletion events. This transitory period gets longer when a bigger batch time window is used, since the number of real-time documents to be deleted is higher and then their deletion requires more time. Anyway, this can mitigated using a better configuration[2] in the serving layer: Elasticsearch allows to define special *client nodes* such that they do not store data but their purpose is just to balance the load of the incoming requests among the *data nodes*.

Concerning the failover scenarios, we analyzed the effects of failures in the data ingestion, in the EventFetcher process, in the real-time and batch layer. The system accomplished the expectations of the Lambda architecture approach: the batch layer resulted to be more fault-tolerant than the real-time one. Having failures in the real-time part provokes drops in the accuracy for the temporary results producted by it. These errors, anyway, will be eventually corrected by the batch results. We also showed how the current implementation of the batch layer recovers from failure events. Having failures in Yarn containers causes the processing time for a batch job to be delayed due to the restart of the application or of a part of it. Unfortunately, the delay in the job execution time can cause a shift in the next scheduled executions, in case it exceeds the batch window length. A reasonable timeout in the Oozie configuration must be set, otherwise subsequent jobs will be considered as failed and they have to be rescheduled. A simple solution could be using Oozie SLA support, which allows to trigger dedicate pipelines in response to a SLA events such as a delayed execution or a failure.

The analyzed failure scenarios cover only a part of all the possible ones, for instance a failure of the master node of the cluster. According to the frameworks used for our implementation, we have a single point of failure both in the real-time and in the batch

[2]https://www.elastic.co/guide/en/elasticsearch/reference/1.6/modules-node.html

layer, even though they have different behaviours. Concerning the batch layer, a failure in Yarn *resource manager* causes the immediate shutdown of all the applications running at that time. In case of HDFS datanode unavailability, Spark jobs cannot be executed as they cannot read input data from HDFS. As regards Oozie, relies on its database availability for reading and executing scheduled executions. A failure in the latter causes the deletion on every schedule batch job. Fortunately, there exists high availability configurations either for Yarn *resource manager*, HDFS *data node* and Oozie database which uses an idle master replica sharing the state of the active one through a storage area network (SAN), a distributed file system or a distributed database. As regards Storm, it relies on Nimbus for scheduling worker processes across the slave nodes in the cluster. In case of its unavailability, the workers continue to work normally and supervisor processes are still able to restart failed workers. However, in case of a slave node failure, tasks will not be restarted on the healthy slave nodes. Anyway, the last release of Storm (0.11) introduced Nimbus high availability[3]. Due to the tight timeframe of this thesis, it was not possible to implement such recovering mechanisms.

[3]https://issues.apache.org/jira/browse/STORM-166

Chapter 6

Conclusions

In this thesis we proposed VimondAnalytics, a web analytics framework based on the Lambda architecture approach. We started from the problem of defining a new scalable, fault-tolerant approach for big data analytics, avoiding to fall into the classic batch or real-time processing. An important and challenging requirement of the system to be defined was the ability to perform both real-time analytics and analysis based on historical data. The Lambda architecture approach fits to this requirement but introduces several drawbacks, mainly due to its complexity.

Part of this work was to simplify the theoretical Lambda architecture idea in order to reduce its complexity. This has been accomplished by bypassing, for each processing layer, the usage of intermediate databases and hence, of partial data views. By implementing the serving layer with a tool providing storage and aggregation features, data are directly ingested into it and aggregated on-the-fly when needed. The key point of this approach is to leverage a real-time and a batch layer in order to reduce the number of documents stored into the serving layer, and consequently, the time required for querying them. Anyway, deleting real-time data when no longer needed does not affect the extensibility of the system. The presence of an immutable data lake allows to define and calculate new aggregations on the entire dataset.

We have shown in Chapter 5 that our approach outperforms a simple real-time-only system in terms of number of documents indexed in the serving layer and of time for executing queries on them. Moreover, by controlling the number of documents stored into the system, we avoid an excessive growth of the query time, which is in average limited by an upper and a lower bound. The boundary values depend on the batch time window setting since longer time windows imply more documents into the system and therefore a slightly higher values, as showed in Section 5.3. The choice of the optimal time window really depends on the use case requirements and on the infrastructure capability of the system.

VimondAnalytics is mainly a proof of concept and several improvements can be done in order to build a production-ready solution. First, high availability mechanisms can be implemented for Storm, Yarn, HDFS and Oozie. This would make the whole system more fault-tolerant and robust. Second, in case of the serving layer implemented with Elasticsearch, an independent visualization layer using Elasticsearch API for interacting with the system can be developed. Even though Kibana offered enough features for this work, it has some limitation since it does not support all the functionalities of Elasticsearch and it is not customizable.

VimondAnalytics also misses a QoS (quality of service) module. For testing purposes, system KPIs are written on CSV files. This cannot be a stable solution as it is important for system administrators to monitor in real-time the functioning of the whole infrastructure. A simple solution would be to send all the metrics collected by the *dropwizard-metrics* library to a *Graphite*[1] or *Ganglia*[2] server. A different solution could be to integrate a 3rd-party module like *Datadog*[3].

[1]http://graphite.wikidot.com
[2]http://ganglia.sourceforge.net
[3]https://www.datadoghq.com

Finally, VimondAnalytics is just the first step in the overall data analysis process and acts as foundation for different services that can be built on top of it. Examples of such services include user profiling and content recommendation, as well as social network integration. VimondAnalytics can be also part of a decision making process, for instance the GQM+S (Goal-Question-Metric + Strategies) [11] defines a framework for aligning business strategies with organizational units within a company. The success of these strategies is evaluated on the evidence given by measurements performed on metrics. VimondAnalytics could provide those metrics for measuring, for instance, the impact of a new product, in terms of number of end-users accessing it.

List of Figures

Bibliography

[1] Apache Flink *"http://flink.apache.org."*

[2] Apache Hadoop *"http://hadoop.apache.org."*

[3] Apache Mesos *"http://mesos.apache.org."*

[4] Apache Oozie *"http://oozie.apache.org."*

[5] Apache Spark *"https://spark.apache.org."*

[6] Apache Storm *"http://storm.apache.org."*

[7] Atzori L., and Iera A. *"The Internet of Things: A survey."*

[8] Armbrust M. et al. *"A view of cloud computing."* Commun. ACM 53, 4 (April 2010), 50-58, 2010. Comput. Netw. 54, 15 (October 2010), 2787-2805, 2010.

[9] Bär N. *"Investigating the Lambda architecture."* University of Zurich, 2014.

[10] BIGTOP-1875 *"https://issues.apache.org/jira/browse/BIGTOP-1875"*

[11] Basili V., Trendowicz A., Kowalczyk M., Heidrich J., Seaman C., Mnch J., and Rombach D. *"Aligning Organizations through Measurement - The GQM+Strategies Approach."* Springer Publishing Company, Incorporated, 2014.

[12] Bonomi F., Milito R., Natarajan P., and Zhu J. *"Fog Computing: A Platform for Internet of Things and Analytics."* Chapter of: Big Data and Internet of Things: A Roadmap for Smart Environments. Springer Publishing Company, Incorporated, 2014.

[13] Boykin O., Ritchie S., OConnell I., and Lin J. *"Summingbird: A framework for integrating batch and online mapreduce computations."* Proceedings of the VLDB Endowment, 7(13), 2014.

[14] Brewer E. *"CAP twelve years later: how the rules have changed."* IEEE Computer, Feb. 2012.

[15] Cooper A. *"A Brief History of Analytics'* JISC CETIS Analytics Series, Vol. 1, No. 9, 2012

[16] Cugola L., and Margara A. *"Processing Flows of Information: From Data Stream to Complex Event Processing."* In Proceedings of the 5th ACM international conference on Distributed event-based system (DEBS '11). ACM, New York, NY, USA, 359-360, 2011.

[17] Dean J., and Ghemawat S. *"MapReduce: Simplified Data Processing on Large Clusters."* Commun. ACM 51, 1 (January 2008), 107-113, 2008

[18] Deloitte Analytics. *"The Analytics Advantage."* 2013

[19] Fourie I., and Bothma T. *"Information seeking: an overview of web tracking and the criteria for tracking software'*, Aslib Proceedings, 2007 (paper)

[20] Elasticsearch *"https://www.elastic.co/products/elasticsearch."*

[21] Garber L. *"Using In-Memory Analytics to Quickly Crunch Big Data."* Computer 45, 10 (October 2012), 16-18, 2012.

[22] Ghemawat S., Gobioff H., and Leung S. *"The Google File System."* In Proceedings of the nineteenth ACM symposium on Operating systems principles (SOSP '03). ACM, New York, NY, USA, 29-43, 2003.

[23] Glass K., and Colbaugh R. *"Web Analytics for Security Informatics."* In Proceedings of the 2011 European Intelligence and Security Informatics Conference (EISIC '11). IEEE Computer Society, Washington, DC, USA, 214-219, 2011.

[24] Google trends. *"http://www.google.com/trends/explore#q=big%20data"*

[25] Gilbert S., and Lynch N. *"Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services."* SIGACT News 33, 2 (June 2002), 51-59, 2012.

[26] HDFS High Availability. *"http://hadoop.apache.org/docs/r2.7.0/hadoop-project-dist/hadoop-hdfs/HDFSHighAvailabilityWithNFS.html"*

[27] Heule S., Nunkesser M., and Hall A. *"HyperLogLog in Practice: Algorithmic Engineering of a State of The Art Cardinality Estimation Algorithm."* In Proceedings of the 16th International Conference on Extending Database Technology (EDBT '13). ACM, New York, NY, USA, 683-692, 2013.

[28] Kambatla K., Kollias G., Kumar V., and Grama A. *"Trends in big data analytics."* Journal of Parallel and Distributed Computing. Volume 74, Issue 7, July 2014, Pages 2561–2573, 2014.

[29] Kelly J. *"Big Data Vendor Revenue and Market Forecast 2013-2017"*, http://wikibon.org/wiki/v/Big_Data_Vendor_Revenue_and_Market_Forecast_2013-2017, 2014.

[30] Kreps J. *"Questioning the Lambda Architecture."* http://radar.oreilly.com/2014/07/questioning-the-lambda-architecture.html, 2014.

[31] Kreps J., Narkhede N., and Rao J. *"Kafka: a Distributed Messaging System for Log Processing."* SIGMOD Workshop on Networking Meets Databases, 2011.

[32] Kulkarni et al. *"Twitter Heron: Stream Processing at Scale."* In Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD '15). ACM, New York, NY, USA, 239-250, 2015.

[33] IMB InfoSphere Streams *"http://www-03.ibm.com/software/products/it/infosphere-streams"*

[34] Laney D. *"3D Data Management: Controlling Data Volume, Velocity, and Variety."* META group Inc., 2001.

[35] Marz N. *"How to beat the cap theorem"*, http://nathanmarz.com/blog/how-to-beat-the-cap-theorem.html, 2011.

[36] Marz N., and Warren J. *"Big Data: Principles and Best Practices of Scalable Realtime Data Systems."* Manning Publications Co., Greenwich, CT, USA, 2015.

[37] McKinsley Global Institute. *"Big data: The next frontier for innovation, competition, and productivity."* Strata summit, 2011.

[38] Narkhede N. *"Real-time stream processing: The next step for Apache Flink."* http://www.confluent.io/blog/real-time-stream-processing-the-next-step-for-apache-flink

[39] Panourgia M., and Kitmeridis N. *"A distributed approach on early trending topics predition on social network services."* Aristotle university of Thessaloniki, 2014.

[40] Russom P. *"Big Data Analytics."* TDWI best practices report, 2011

[41] Singh D., and Reddy C. K. *"A survey on platforms for big data analytics."* Journal of Big Data, vol. 2, article 8, 2014.

[42] Stonebraker M., Çetintemel U., and Zdonik S. *"The 8 requirements of real-time stream processing."* SIGMOD Rec. 34, 4 (December 2005), 42-47, 2005.

[43] Toshniwal A. et al. *"Storm@twitter"*. In Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data (SIGMOD '14). ACM, New York, NY, USA, 147-156, 2014.

[44] Vavilapalli V. K. et al. *"Apache Hadoop YARN: Yet Another Resource Negotiator."* In Proceedings of the 4th annual Symposium on Cloud Computing (SOCC '13). ACM, New York, NY, USA, , Article 5 , 16 pages, 2013.

[45] Vimond Media Solution. *"http://www.vimond.com."*

[46] Ward J. S., and Barker A. *"Undefined by data: A survey of Big Data Definitions."* CoRR, vol. abs/1309.5821, 2013.

[47] Web Analytics Association. *"Web Analytics Definitions."* 2008.

[48] Zaharia M. et al. *"Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing."* In Proceedings of the 9th USENIX conference on Networked Systems Design and Implementation, 2012.

[49] Zaharia M. et al. *"Spark: Cluster Computing with Working Sets."* In Proceedings of the 2nd USENIX conference on Hot topics in cloud computing (HotCloud'10). USENIX Association, Berkeley, CA, USA, 10-10, 2010.